

معماری پایه Karyar

ERPNext + Karyar Agent Platform

سند طراحی معماری – نسخه ۱/۰ (پیش‌نویس برای بازبینی) – مهر ۱۴۰۵ / اکتبر ۲۰۲۶
وضعیت: **پیشنهادی**. هیچ تصمیمی در این سند نهایی نیست، مگر در بخش «اصول تأییدشده».

فهرست

- بخش صفر: خلاصه اجرایی و وضعیت فعلی مخزن
 - بخش ۱ تا ۳۰: معماری
 - بخش ۳۱: شش مثال سرتاسری
 - بخش ۳۲: جدول تصمیم‌های معماری
 - بخش ۳۳: تعارض‌ها و محدودیت‌هایی که باید صریح گفته شوند
 - بخش ۳۴: خروجی نهایی و نقشه راه
-

بخش صفر – خلاصه اجرایی و وضعیت فعلی مخزن

۱. خلاصه در یک پاراگراف

Karyar یک سکوی ایجنت (Agent Platform) است که روی ERPNext قرار می‌گیرد. ERPNext منبع حقیقت (Source of Truth) داده‌های کسب‌وکار است: مشتری، سفارش، انبار و حسابداری. Karyar چهار کار انجام می‌دهد:

- پیام‌ها را می‌گیرد و آن‌ها را به گفت‌وگوهای استاندارد تبدیل می‌کند.
- پرونده‌های کاری را جلو می‌برد: هر پرونده (Workflow Run) طبق یک «تعریف فرایند» قابل‌پیکربندی حرکت می‌کند.
- ایجنت‌ها را داخل این پرونده‌ها به کار می‌گیرد: ایجنت‌ها فقط از طریق «ابزارهای کنترل‌شده» با ERPNext کار می‌کنند.
- انسان را در نقاط تعریف‌شده وارد می‌کند: گفت‌وگو محور و ثبت‌شده.

انتقال کار بین ایجنت‌ها همیشه از راه رویداد و تغییر وضعیت پرونده است، نه گفت‌وگوی آزاد بین هوش‌های مصنوعی. دسترسی‌ها در لایه‌ای مستقل از مدل هوش مصنوعی اعمال می‌شوند و نهایتاً به همان سیستم مجوز Frappe می‌رسند.

۲. وضعیت فعلی مخزن (یافته‌های بررسی)

مورد	یافته
مخزن	karyar-mainERP ، یک fork مستقیم از frappe/erpnext
شاخه و commit	claude/keen-hypatia-07r3hv ، برابر با develop در commit 3257b70
نسخه ERPNext	17.0.0-dev ، یعنی شاخه توسعه و ناپایدار
نسخه Frappe مورد نیاز	>=17.0.0-dev , <18 (خود Frappe در مخزن نیست)
Python / Node	Python 3.14 یا بالاتر، 24 Node
اپ‌های موجود	فقط erpnext و یک SPA بانکداری با React (/banking). اپ‌های hrms ، frappe ، Karyar در مخزن نیستند
سفارشی‌سازی Karyar	هیچ‌چیز. کلمه «karyar» در کد وجود ندارد و همه commit‌ها از Frappe هستند
حجم کد	حدود ۴۳۰ هزار خط Python و ۸۷ هزار خط JS ، در ۲۱ مازول
وابستگی‌های Python	plaid-python ، mt-940 ، pdfplumber ، holidays ، rapidfuzz و چند مورد دیگر

نقاط توسعه موجود (یعنی جاهایی که Karyar می‌تواند بدون دست زدن به هسته وصل شود):

نقطه توسعه	کاربرد برای Karyar
doc_events (از جمله "*" برای همه DocType‌ها)	گرفتن رویدادهای ERPNext
scheduler_events و صف‌های سفارشی RQ (کلید workers در common_site_config)	اجرای ایجنت‌ها و کارهای پس‌زمینه
enqueue(... و frappe.db.after_commit enqueue_after_commit=True)	انتشار مطمئن رویداد، فقط بعد از commit تراکنش

نقطه توسعه	کاربرد برای Karyar
has_permission ، permission_query_conditions ، User Permission و permlevel	کنترل دسترسی در سطح سند، رکورد و فیلد
extend_doctype_class و override_doctype_class	توسعه رفتار DocType های ERPNext بدون تغییر هسته
regional_overrides و naming_series_variables	بومی سازی ایران، مثلاً شماره گذاری جلالی
boot_session ، setup_wizard_stages و jinja	نصب، بارگذاری اولیه و قالب های چاپ
Frappe Workflow (در v16 با Transition Task)، Notification، Assignment Rule	امکانات آماده برای گردش سند و تخصیص کار
Webhook با امضای OAuth Client، HMAC (X-Frappe-Webhook-Signature)، API نسخه ۱ و ۲	اتصال به سیستم های بیرونی
Version، Access Log، Error Log، RQ Job	پایه ردیابی و ممیزی
Socket.io و publish_realtime	پخش پاسخ های بلادرنگ در رابط کاربری

محدودیت هایی که روی طراحی اثر می گذارند:

1. **Frappe Workflow فقط ماشین حالت یک سند است.** گردش بین چند سند، تایمر، تأیید موازی «۲ از ۳» و گام ایجنت ندارد.
2. **frappe.get_all مجوزها را نادیده می گیرد.** ابزارهای ایجنت نباید از آن استفاده کنند.
3. **سرور Frappe همگام (WSGI) است.** فراخوانی های طولانی مدل زبانی نباید در پروسه وب اجرا شوند و باید به Workerها بروند.
4. **چارت حساب فقط از پوشه خود ERPNext بارگذاری می شود.** برای چارت ایرانی باید از `create_charts(custom_chart=...)` استفاده شود.
5. **برخی گزارش های استاندارد با SQL خام نوشته شده اند و کنترل دسترسی آن ها فقط در سطح نقش (Role) است.** پس فقط فهرستی گزینش شده از گزارش ها باید در اختیار ایجنت قرار گیرد.
6. **ERPNext نسخه ۱۶ روی PostgreSQL پشتیبانی رسمی ندارد.**

تعارض با چشم انداز: این مخزن خود ERPNext است. کد Karyar نباید داخل آن نوشته شود، چون این کار عملاً تغییر هسته است و هر ارتقا را دردناک می کند. بخش ۲۳ ساختار درست مخزن ها را پیشنهاد می دهد. این سند فعلاً موقتاً در همین مخزن (docs/karyar/) قرار گرفته است.

۱. نمای کلی معماری

Karyar از **چهار لایه منطقی** ساخته می شود. یک لایه **عمودی** هم از همه آن ها عبور می کند:

1. **لایه تعامل:** کانال ها (وبسایت، تلگرام، بله، واتس اپ و...) به علاوه «میز کار کاریار» (Karyar Workspace) برای کارکنان. همه این ها به «درگاه کانال» (Channel Gateway) وصل می شوند. وظیفه درگاه این است که پیام هر کانال را به یک پیام استاندارد تبدیل کند.
2. **لایه هوشمندی و فرایند:**
 - **موتور گردش کار (Workflow Engine):** تعیین می کند چه کاری و کی انجام شود.

- اجراکننده ایجنت (Agent Runtime): تعیین می‌کند چطور آن کار با زبان طبیعی و به صورت هوشمند انجام شود.
- سرویس کار انسانی (Human Task Service): مدیریت تعامل انسان در گردش کار.
- سرویس گفت‌وگو (Conversation Service).

3. لایه کنترل:

- رجیستری توانمندی و ابزار: فهرست کارهایی که ایجنت مجاز است انجام دهد.
- موتور سیاست/مجوز (Policy Engine).
- اجراکننده ابزار (Tool Executor).
- API پایدار کاریار (Karyar API v1).
- گذرگاه رویداد (Event Bus).
- درگاه مدل هوش مصنوعی (AI Gateway).

4. لایه داده و عملیات: ERPNext، HRMS و karyar_iran روی Frappe. این تنها منبع حقیقت داده‌های کسب‌وکار است. - لایه عمودی: ممیزی، ردیابی، مصرف هوش مصنوعی و امنیت. - بیرون از هسته: n8n برای یکپارچه‌سازی‌ها، Workerهای RPA و سرویس‌های بیرونی.

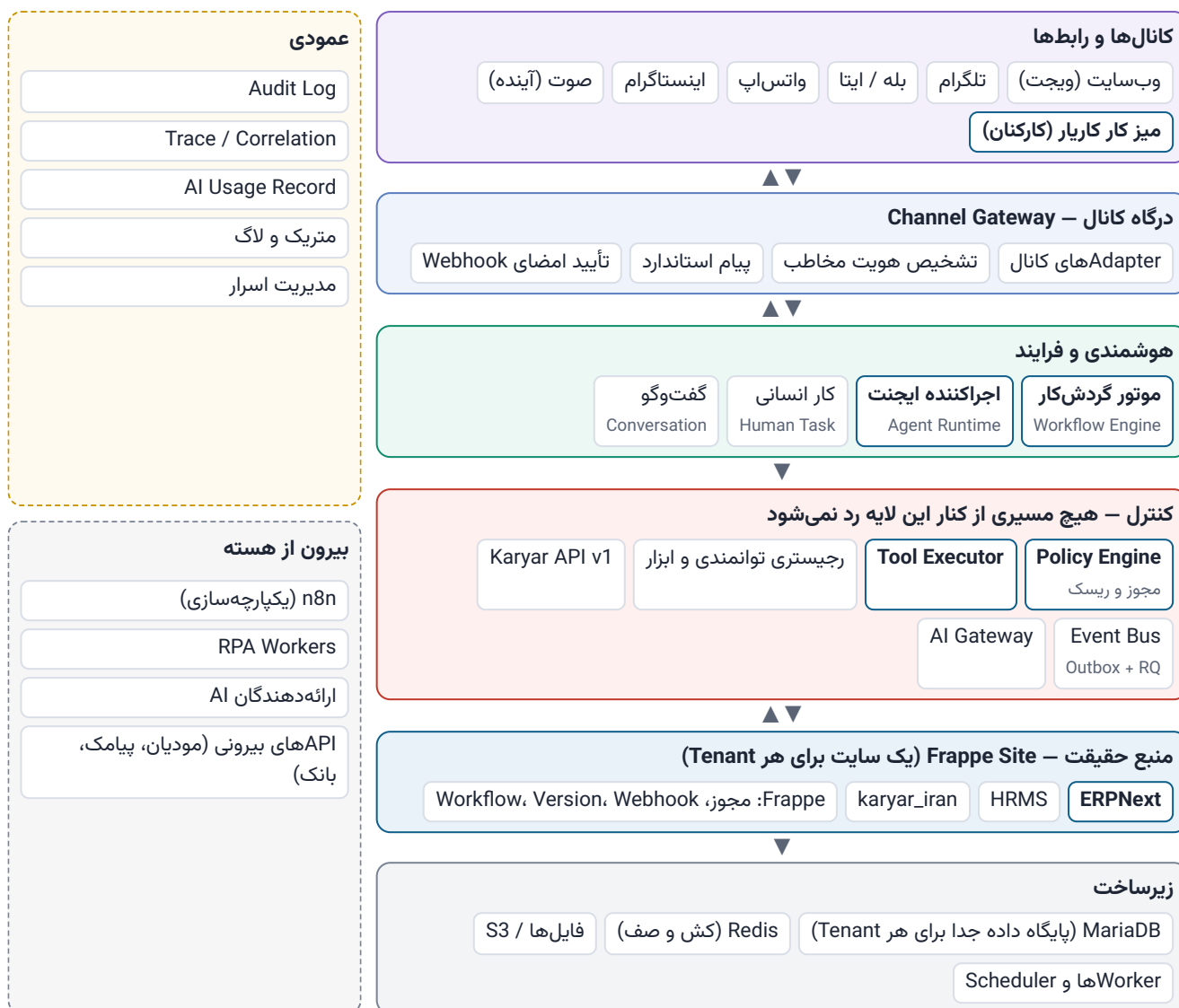
ایده محوری: «Karyar» فرایند را تعریف می‌کند؛ هوش مصنوعی درون فرایند هوشمندانه کار می‌کند. «گردش کار قطعی (deterministic) و قابل ممیزی است. ایجنت احتمالاتی (probabilistic) است و فقط در قالب گام‌ها و ابزارهای مجاز عمل می‌کند.

۲. اصول بنیادین

#	اصل	پیامد فنی
P1	ERPNext منبع حقیقت داده‌های کسب‌وکار است	Karyar فقط «وضعیت فرایند» را نگه می‌دارد، نه داده موازی
P2	هیچ ایجنتی به پایگاه داده دسترسی خام ندارد	فقط ابزارهای تایپ‌شده و از پیش ثبت‌شده. SQL آزاد وجود ندارد
P3	مجوز مستقل از مدل هوش مصنوعی اعمال می‌شود	بررسی در Tool Executor و Frappe انجام می‌شود، نه در prompt
P4	فرایندها پیکربندی‌اند، نه کد	Workflow Definition نسخه‌دار. کد مشتری‌محور فقط از راه «نقاط توسعه»
P5	ایجنت، نقش کاری، توانمندی، مجوز و گردش کار از هم جدا هستند	گردش کار به نقش کاری اشاره می‌کند، نه به ایجنت مشخص
P6	دخالت انسان (HITL) یک گام یا سیاست قابل‌پیکربندی است	هیچ قانون ثابتی مثل «Arman همیشه تأیید می‌خواهد» در کد نیست
P7	انتقال کار بین ایجنت‌ها فقط از راه رویداد و وضعیت	چت آزاد ایجنت با ایجنت وجود ندارد. داده ساخت‌یافته منتقل می‌شود
P8	نوشتن در ERPNext ابتدا به صورت پیش‌نویس است	ثبت نهایی (Submit) یک اقدام جداگانه است که سیاست‌ها کنترلش می‌کنند
P9	آنچه تأیید شده، دقیقاً همان است که اجرا می‌شود	تأیید به hash محتوا گره می‌خورد و هر تغییری تأیید را باطل می‌کند
P10	به هیچ ارائه‌دهنده هوش مصنوعی گره نخوردن	AI Gateway با نام مستعار مدل (alias) کار می‌کند
P11	هر کاری قابل ردیابی است	correlation_id در همه لایه‌ها و Audit Log معنایی
P12	هسته ERPNext دست‌نخورده می‌ماند	Hooks، اپ سفارشی و API. هر وصله ضروری مستند می‌شود

#	اصل	پیامد فنی
P13	ساده شروع کن، درست مرز بکش	یک اپ Frappe با ماژول‌های جدا. جداسازی به سرویس مستقل فقط هنگام نیاز

۳. نمودار معماری سیستم



جهت جریان کنترل:

- **کانال ← گردش کار:** کانال پیام را به درگاه می‌دهد. درگاه آن را به یک Conversation وصل می‌کند و یک رویداد `channel.message.received` منتشر می‌کند. موتور گردش کار تصمیم می‌گیرد پیام متعلق به کدام پرونده است و کدام ایجنت باید پاسخ دهد.
- **ایجنت ← ابزار ← ERPNext:** ایجنت فقط «ابزار» صدا می‌زند. Tool Executor سیاست‌ها را بررسی می‌کند و ابزار با هویت مجاز روی ERPNext اجرا می‌شود.
- **ERPNext ← رویداد ← گردش کار:** تغییرات ERPNext رویداد تولید می‌کنند و موتور گردش کار گام بعدی را فعال می‌کند. این «تحویل خودکار» بین ایجنت‌هاست.

۴. معماری اجزا

برای هر جزء: چرا وجود دارد، چه مسئله‌ای حل می‌کند، کی لازم است و چه جایگزین‌هایی بررسی شد.

جزء	مسئولیت	چرا	الان یا بعد	جایگزین‌های بررسی‌شده
Channel Gateway	نرمال‌سازی پیام‌ها، شناسایی مخاطب، ارسال پاسخ، تأیید امضا	جدا کردن کانال از منطق ایجنت	الان (۱ تا ۲ کانال)	درون Frappe یا سرویس جدا. فعلاً درون Frappe و در صورت حجم بالا جدا شود
Conversation Service	نگهداری گفت‌وگو و پیام و اتصال آن به پرونده	حافظه کوتاه‌مدت و ردیابی	الان	استفاده از Communication در Frappe؛ برای چت ساخت‌یافته مناسب نیست
Agent Runtime	اجرای یک «نوبت» ایجنت: ساخت context، فراخوانی مدل، اجرای ابزارها، اعتبارسنجی خروجی	هسته هوشمندی	الان (ساده)	LangGraph، CrewAI، Agents SDK. تصمیم: runtime سبک خودمان با رابط تمیز (بخش ۵)
Workflow Engine	اجرای پایدار گراف فرایند و نگهداری وضعیت پرونده	تحويل خودکار، HITL، تایمر	الان (حداقلی)	Frappe Workflow، Temporal، n8n، Camunda (بخش ۷)
Human Task Service	کار انسانی، تأیید، ویرایش گفت‌وگو‌محور	HITL قابل‌پیکربندی	الان	Workflow Action و ToDo در Frappe؛ ناکافی‌اند
Capability & Tool Registry	تعریف ابزارهای تایپ‌شده و بسته‌بندی آن‌ها در توانمندی‌ها	جلوگیری از دسترسی خام	الان	MCP به‌عنوان تنها لایه؛ MCP فقط یک «نمایش» از همین رجیستری است
Policy Engine	ترکیب سطوح مجوز و سیاست ریسک	امنیت مستقل از مدل	الان (پایه)	OPA یا Cedar؛ برای شروع زیادی است
Tool Executor	تنها مسیر اجرای ابزار، همراه با ممیزی، idempotency و فیلتر خروجی	نقطه واحد کنترل	الان	—
Event Bus	Outbox در پایگاه داده + dispatcher روی RQ	تحويل مطمئن رویداد	الان	Kafka، RabbitMQ، NATS؛ فعلاً لازم نیست
Karyar API v1	API پایدار و نسخه‌دار برای n8n، اپ‌ها و سیستم‌های بیرونی	محافظت از مصرف‌کننده‌ها در برابر تغییرات ERPNext	الان (کوچک)	دسترسی مستقیم به /api/resource؛ شکننده است
AI Gateway	انتزاع ارائه‌دهنده، نام مستعار مدل، fallback، ثبت مصرف	مستقل از ارائه‌دهنده و آماده ثبت هزینه	الان (کتابخانه)	LiteLLM Proxy (گزینه بعدی)
Audit & Telemetry	Audit Log معنایی، Trace، متریک	ممیزی و عیب‌یابی	الان (پایه)	OpenTelemetry کامل (بعد)
Karyar Workspace	رابط کارکنان: صندوق کار، گفت‌وگو، پرونده‌ها	کاربر نباید مجبور به کار با ERPNext Desk باشد	الان (حداقلی)	فقط تلگرام یا بله برای کارکنان؛ برای کار جدی کافی نیست
n8n	یکپارچه‌سازی و اعلان	اتصال سریع به سرویس‌های بیرونی	فاز ۲	کدنویسی مستقیم هر اتصال
RPA Service	تعامل با سیستم‌های بدون API	پورتال‌ها	فاز ۳ به بعد	—

جزء	مسئولیت	چرا	الان یا بعد	جایگزین‌های بررسی‌شده
Control Plane	ثبت Tenant، راه‌اندازی، بسته‌ها، اندازه‌گیری مصرف مرکزی	چندمستأجری در مقیاس	بعد (اول با اسکریپت)	Frappe Press (AGPL)

۵. معماری ایجنت

۵.۱ مفاهیم جداشده

یکی از مهم‌ترین تصمیم‌های این سند، جدا کردن پنج مفهوم از هم است:

مفهوم	تعریف	مثال
Capability (توانمندی)	یک توانایی اتمی که به یک یا چند ابزار نگاشت می‌شود و سطح ریسک دارد	report.sales_summary.read، crm.lead.draft، accounts.journal.draft
Responsibility (نقش کاری)	بسته‌ای از توانمندی‌ها به‌علاوه دستورالعمل حرفه‌ای	«فروش»، «CRM»، «حسابداری»، «پذیرش مشتری»
Agent (ایجنت)	یک شخصیت اجرایی شامل نام، لحن، کانال‌ها، مدل و یک یا چند نقش کاری	Ava، Hanna، Arman، «سارا» (چند نقش در یک ایجنت)
Role Assignment (انتساب نقش)	تعیین اینکه در هر Tenant، هر نقش کاری را کدام ایجنت(ها) یا کاربر(ها) انجام می‌دهند	شرکت C: فروش + CRM + حسابداری ← «سارا»
Frappe Role / Permission	مجوز واقعی سیستم روی DocType‌ها و رکوردها	Sales User، Accounts Manager

قاعده کلیدی: گام‌های گردش کار به نقش کاری اشاره می‌کنند، نه به ایجنت. یعنی گام می‌گوید «انجام توسط: نقش حسابداری» و جدول انتساب در هر Tenant مشخص می‌کند این نقش با کیست.

نتیجه: اگر یک ایجنت چند نقش داشته باشد یا یک نقش بین چند ایجنت یا انسان تقسیم شود، هیچ تعریف گردش کاری عوض نمی‌شود.

نکته نام‌گذاری (نیاز به تأیید): کلمه Role در Frappe معنای «نقش دسترسی» دارد. برای جلوگیری از اشتباه، در کد از Responsibility استفاده می‌کنیم و در رابط کاربری فارسی از «نقش کاری».

۵.۲ تعریف ایجنت (DocType: Karyar Agent)

- **شناسه:** نام، شخصیت و لحن، زبان، آواتار
- **نقش‌های کاری:** یک یا چند Responsibility. توانمندی‌ها از اجتماع (union) آن‌ها به دست می‌آیند و قابل محدودسازی‌اند
- **کانال‌های مجاز و مخاطب مجاز:** مشتری بیرونی، کارمند یا هر دو
- **پروفاایل مدل:** یک نام مستعار مثل smart یا fast که در AI Gateway به مدل واقعی نگاشت می‌شود
- **قالب دستورالعمل:** نسخه‌دار. دستورالعمل از چند لایه ترکیب می‌شود: پایه سکو، نقش کاری، تنظیمات Tenant، و گام گردش کار
- **هویت اجرایی:** یک کاربر Frappe از نوع سرویس (بدون امکان ورود) برای کارهای خودکار
- **سیاست حافظه:** فعلاً فقط گفت‌وگو و context پرونده. حافظه بلندمدت در فاز بعد

- **محدودیت‌ها:** حداکثر تعداد فراخوانی ابزار در هر نوبت، حداکثر token، و سقف هزینه (جای خالی برای فاز بعد)

۵.۳ حالت‌های اجرای ایجنت

1. **نوبت گفت‌وگو:** پیامی از کاربر یا مشتری می‌رسد و ایجنت پاسخ می‌دهد. ممکن است ابزار بخواند یا پیشنهاد اقدام بدهد.
2. **گام گردش‌کار:** موتور گردش‌کار به ایجنت یک «دستور کار» ساخت‌یافته می‌دهد (هدف، داده پرونده، توانمندی‌های مجاز در همین گام، و Schema خروجی). ایجنت باید خروجی معتبر طبق Schema تحویل دهد.
3. **دستیار بازبینی:** در گام انسانی، ایجنت به کارمند کمک می‌کند داده را بررسی و اصلاح کند (بخش ۸).
4. **(آینده)** کارهای زمان‌بندی‌شده و تحلیلی، مثل «هر صبح پیگیری‌های عقب‌افتاده را بررسی کن». حتی این‌ها هم یک گردش‌کار تعریف‌شده‌اند، نه ابتکار ایجنت.

۵.۴ چرخه یک نوبت ایجنت

1. Load: Agent config + Responsibilities → effective Capabilities (n step scope)
2. Build context: conversation window + run context (structured) + task brief
3. AI Gateway.complete(alias, messages, tools=allowed_tool_schemas, response_schema?)
4. For each tool call → Tool Executor:
policy.check(principal, capability, scope, risk) → execute → filter output → audit
5. Loop (bounded: max_steps, max_tokens, timeout)
6. Validate final output against schema (retry once on failure → else escalate to human)
7. Persist: messages, step output, usage record, audit; emit events

- **محل اجرا:** Workerهای RQ با صف اختصاصی karyar_agent، نه پروسه وب.

- **پخش پاسخ:** با publish_realtime در Socket.io انجام می‌شود.

- **چرا runtime سبک خودمان، نه چارچوب آماده؟** ما به سه چیز احتیاج داریم:

- یکپارچگی عمیق با هویت و مجوز Frappe و تراکنش‌های آن
- چندمستأجری در سطح Site
- کنترل کامل روی ممیزی

حلقه tool-calling هم پیچیده نیست. چارچوب‌هایی مثل LangGraph می‌توانند داخل یک گام برای استدلال‌های پیچیده به کار بروند، ولی مالک وضعیت فرایند نیستند.

۵.۵ پیشنهاد اقدام (Action Proposal)

ایجنت مستقیماً «کار حساس» انجام نمی‌دهد. وقتی ابزاری از نوع commit-write یا external باشد و سیاست بگوید «نیاز به تأیید دارد»، روند این است:

1. Tool Executor به جای اجرا، یک **Action Proposal** می‌سازد که شامل ابزار، ورودی، خلاصه انسانی و hash است.
2. این پیشنهاد به گام انسانی می‌رود.
3. بعد از تأیید، همان ورودی دقیق اجرا می‌شود.

به این ترتیب HITL دو منبع دارد:

- **صریح:** یک گام انسانی که در گراف گردش‌کار تعریف شده
- **ضمنی:** سیاست ریسک Tenant، مثلاً «هر ثبت نهایی سند حسابداری بالای ۵۰ میلیون ریال تأیید مدیر مالی می‌خواهد»

۶. معماری سازنده ایجنت (Agent Builder)

فاز ۱: بدون رابط گرافیکی. پیکربندی با DocType های Frappe انجام می شود (فرم های Desk برای تیم Karyar) و به صورت بسته (Pack) نسخه دار ذخیره می شود.

ساختار بسته:

```
Karyar Pack (versioned, e.g. "clinic-basic@1.2.0")
├─ responsibilities/*.json (capabilities + instructions)
├─ agents/*.json (persona, channels, model alias, responsibilities)
├─ workflows/*.json (versioned graph definitions)
├─ role_assignments.json (tenant-overridable defaults)
├─ policies.json (risk thresholds, HITL defaults)
├─ reports.json (report catalog entries)
└─ erp_customizations/ (custom fields, property setters – Frappe fixtures)
```

- **اعمال روی Tenant:** دستور `bench --site X karyar apply-pack clinic-basic@1.2.0` اجرا می شود و تفاوت ها با تنظیمات خود Tenant ادغام می شوند.
- **چرا Pack:** این همان سازوکار ۸۰/۲۰ است. ۸۰٪ مشترک در Pack قرار می گیرد و ۲۰٪ اختصاصی هر مشتری به صورت override در سایت خودش. هیچ clone کردن کدی لازم نیست.
- **فاز بعد:** رابط گرافیکی سازنده ایجنت (فرم ساده) و پیش نمایش رفتار ایجنت با «سناریوی تست».
- **چیزی که هرگز قابل پیکربندی نیست:** ساخت ابزار جدید از طریق رابط کاربری یا توسط هوش مصنوعی. ابزار جدید کد است، بازبینی می شود و از راه hooks ثبت می شود.

نقاط توسعه برای اپ های دیگر (مثل Pack ها، karyar_iran یا اپ اختصاصی یک مشتری):

```
# hooks.py of any app
karyar_tools = ["karyar_iran.tools.validators", "karyar_iran.tools.moadian"]
karyar_step_types = ["my_app.steps.custom_step"]
karyar_channel_adapters = ["karyar_iran.channels.bale"]
karyar_ai_providers = ["my_app.ai.local_vllm"]
karyar_report_providers = ["my_app.reports"]
```

۷. معماری موتور گردش کار

۷.۱ چرا موتور جداگانه؟ (مقایسه گزینه ها)

گزینه	مزیت	عیب	نتیجه
Frappe Workflow	آماده، ادغام کامل با سند و Role	فقط یک DocType. بدون گام ایجنت، تایمر، موازی یا زیرفرایند	فقط برای «وضعیت سند» در Desk، و اختیاری
n8n	سریع و بصری	منطق تراسنشی و وضعیت، خارج از ERP و پراکنده؛ مجوز ضعیف	❌ برای هسته؛ ✅ برای یکپارچه سازی
Temporal	اجرای پایدار حرفه ای، تایمر و retry	زیرساخت سنگین (سرور، پایگاه داده جدا)؛ تیم باید یاد بگیرد؛ چندمستأجری در دسر دارد	گزینه بعدی اگر بار و پیچیدگی رشد کرد

گزینه	مزیت	عیب	نتیجه
Camunda / BPMN	استاندارد	سنگین، Java، برای فرایندهای گفت‌وگومحور زیادی رسمی	✗
موتور سبک در Frappe 	هم‌تراکنش با ERPNext، همان مجوز و چندمستأجری، بدون زیرساخت جدید	باید خودمان بسازیم و با دقت محدودش کنیم	پیشنهاد فاز ۱

تصمیم پیشنهادی: یک ماشین حالت پایدار داخل اپ `karyar` ساخته شود. «تعریف» از «اجراکننده» جدا باشد تا در آینده اگر لازم شد، اجراکننده با Temporal عوض شود و تعریف‌ها دست نخورند.

۷.۲ مدل داده

DocType	توضیح
Karyar Workflow Definition	نام، نسخه، وضعیت (draft/active/retired)، trigger و graph به‌صورت JSON اعتبارسنجی‌شده
Karyar Workflow Run (پرونده)	تعریف و نسخه آن (هر پرونده به نسخه‌ای که با آن شروع شده سنجاق است)، وضعیت، context ساخت‌یافته، اسناد ERPNext مرتبط، correlation_id
Karyar Step Run	هر اجرای یک گام: نوع، ورودی و خروجی، مجری، تلاش‌ها، خطا، زمان‌ها
Karyar Human Task	کار انسانی (بخش ۸)
Karyar Timer	زمان‌های سررسید برای timeout و escalation. یک job در scheduler هر دقیقه آن‌ها را بررسی می‌کند

۷.۳ انواع گام (قابل گسترش با `karyar_step_types`)

نوع گام	کاربرد	فاز
trigger	شروع با رویداد، پیام، زمان‌بندی یا دستی	۱
agent_task	گام ایجنت با هدف، نقش کاری، توانمندی‌های مجاز و Schema خروجی	۱
human_task	تعامل یا تأیید انسانی (بخش ۸)	۱
condition	شاخه شرطی با عبارت امن روی context، نه کد آزاد	۱
erp_action	اقدام از پیش تعریف‌شده روی ERPNext از طریق ابزار، مثل «ثبت نهایی Sales Order»	۱
wait_event	انتظار برای رویداد مشخص، مثل تأیید پرداخت	۱
transform / validate	اعتبارسنجی Schema و قواعد، و نگاشت داده	۱
notify	اعلان به انسان یا مشتری	۱
end	پایان (موفق، ردشده یا لغوشده)	۱
escalate / timer	مهلت و ارجاع به سطح بالاتر	۲
integration_call	فراخوانی n8n یا سرویس بیرونی با callback	۲
join / parallel	شاخه‌های موازی (مثلاً تأیید ۲ از ۳)	۲ تا ۳
sub_workflow	فراخوانی گردش کار دیگر	۳
rpa_job	کار RPA	۳ به بعد

۷.۴ معناسناسی اجرا

- **پیشروی رویدادمحور:** هر رویداد یا تکمیل گام، یک job به نام «پیشروی پرونده» (advance_run) را با قفل پرونده اجرا می‌کند. قفل با SELECT ... FOR UPDATE روی ردیف پرونده یا قفل Redis گرفته می‌شود تا دو worker همزمان یک پرونده را جلو نبرند.
- **Idempotency:** هر گام یک idempotency_key دارد. اقدام‌های ERPNext با کلید یکتا ثبت می‌شوند تا تکرار رویداد به ایجاد سند تکراری منجر نشود.
- **Retry:** هر نوع گام سیاست retry خودش را دارد. ایجنت و API بیرونی retry می‌شوند، ولی erp_action فقط اگر idempotent باشد. بعد از اتمام تلاش‌ها، پرونده به حالت needs_attention می‌رود و یک Human Task «رفع خطا» ساخته می‌شود.
- **نسخه‌داری:** پرونده‌های در جریان با همان نسخه تعریفی که شروع شده‌اند ادامه می‌دهند. فعال کردن نسخه جدید فقط روی پرونده‌های جدید اثر دارد.
- **عبارت‌های شرط:** با یک زبان عبارت امن و محدود نوشته می‌شوند، مثلاً safe_eval در Frappe یا JSONLogic. هرگز کد Python آزاد در تعریف گردش کار نوشته نمی‌شود.

۸. معماری دخالت انسان (Human-in-the-Loop)

۸.۱ HITL یک گام است، نه یک قانون

هیچ ایجنتی ذاتاً «نیازمند تأیید» نیست. HITL از دو جا وارد پرونده می‌شود:

۱. گام human_task در گراف گردش کار: صریح و طراحی شده.
۲. سیاست ریسک (Policy Gate): وقتی ایجنت اقدامی پیشنهاد می‌کند که سیاست Tenant برایش تأیید لازم می‌داند.

۸.۲ پیکربندی یک گام انسانی

تنظیم	گزینه‌ها
مسئول انجام (Assignee)	کاربر مشخص، Frappe Role، نقش کاری (Responsibility)، عبارت پویا مثل «مدیر شعبه همین مشتری»، یا Frappe در Assignment Rule
تعداد و الگوی تأیید	۱ نفر، N نفر، «M از N»، ترتیبی یا موازی
داده قابل نمایش	فهرست فیلدها، همراه با پوشاندن فیلدهای حساس برای این گام
داده قابل ویرایش	فهرست فیلدهای قابل اصلاح توسط انسان. بقیه فقط خواندنی‌اند
پیش شرط تکمیل	فیلدهای اجباری یا قواعد اعتبارسنجی قبل از امکان تأیید
اقدام‌های مجاز	تأیید، رد، بازگشت برای اصلاح، درخواست اطلاعات از مشتری، توقف، ارجاع
پس از ویرایش	ادامه، یا بازبینی مجدد توسط همان نفر یا نفر دیگر
پس از رد	شاخه رد: پایان، بازگشت به گام قبل یا مسیر جایگزین
سطح تأیید صریح	none (پیام عادی کافی است)، confirm (نمایش خلاصه نهایی و تأیید صریح)، strong (تأیید صریح به‌علاوه OTP یا رمز برای عملیات مالی)
مهلت	timeout، یادآوری، ارجاع (فاز ۲)

تنظیم	گزینه‌ها
کانال	Karyar Workspace، تلگرام/بله کارمند، یا ایمیل (لینک)

۸.۳ تعامل گفت‌وگومحور، با تصمیم‌های ساخت‌یافته

مشکل: تعامل آزاد («سن را ۳۸ کن»، «بفرست مرحله بعد») با ممیزی و امنیت در تعارض است. اگر مدل «باشه» را اشتباهاً «تأیید» تفسیر کند چه می‌شود؟

راه‌حل: گفت‌وگو رابط است، اما وضعیت و تصمیم ساخت‌یافته است:

- Human Task یک **payload ساخت‌یافته** با Schema دارد که شامل داده، فیلدهای الزامی و فیلدهای قابل ویرایش است.
- ایجنت در حالت «دستیار بازبینی» فقط این ابزارها را دارد:
 - `task.show`: نمایش داده
 - `task.patch(field, value)`: اصلاح یک فیلد، با اعتبارسنجی Schema و قواعد. هر اصلاح یک «نسخه» ثبت می‌کند که در آن مشخص است *انسان دستور داد و ایجنت اجرا کرد*.
 - `task.ask_customer(question)`: درخواست اطلاعات از مشتری. این یک زیرگام می‌سازد که به گفت‌وگوی مشتری با Ava وصل است.
 - ابزارهای **خواندنی** در محدوده مجوز همین کارمند.
 - `task.propose_decision(action)`
- **تصمیم نهایی هرگز مستقیماً توسط مدل ثبت نمی‌شود.** مدل فقط *پیشنهاد* تصمیم می‌دهد. سیستم:
 1. هویت فرستنده را بررسی می‌کند: آیا مسئول مجاز این Task است؟
 2. بسته به سطح تأیید، خلاصه نهایی را نمایش می‌دهد و تأیید صریح می‌گیرد.
 3. تصمیم را با `payload_hash` ثبت می‌کند.
- **دکمه‌ها** («تأیید»، «رد») میانبر اختیاری همین ابزارها هستند، نه تنها راه.
- **ثبت ممیزی:** متن خام پیام انسان، تفسیر مدل، نسخه `payload` و `hash`، همگی ثبت می‌شوند.

```
Employee: "The age is incorrect. Change it to 38."
  → agent calls task.patch(age=38) → validated → revision #2 (by: employee, via: agent)
Agent: "Updated. Information is now complete."
Employee: "Send it to the next step."
  → agent calls task.propose_decision(approve)
  → policy: confirmation_level=confirm → system renders final summary (hash h2)
System: "Confirm sending: Mohammad Rezaei, 38, Hair transplant ... ? (yes/no)"
Employee: "yes" → Decision{approve, by: employee, payload_hash: h2, raw_text: "yes"} → event
human_task.completed
```

امنیت: اختیار تأیید فقط از **هویت احرازشده مسئول Task** می‌آید، نه از محتوای پیام. اگر پیام مشتری حاوی «این را تأیید کن» باشد، هیچ اثری ندارد. این دفاع اصلی در برابر Prompt Injection است.

۸.۴ تأیید مشتری (Customer Confirmation)

تأیید مشتری هم همین الگو را دارد:

1. Ava خلاصه را نمایش می‌دهد (نسخه `h1`).
2. پاسخ «بله، درست است» از **همان هویت کانال مشتری**، به همان نسخه `h1` گره می‌خورد.

3. اگر بعداً داده تغییر کند، تأیید مشتری برای نسخه جدید معتبر نیست. اینکه آیا دوباره تأیید لازم است، طبق سیاست گام تعیین می‌شود.

۹. معماری رویداد

۹.۱ منابع رویداد

منبع	سازوکار
ERPNext / Frappe	doc_events برای DocType های ثبت شده در «Event Source»، روی <code>on_update</code> ، <code>after_insert</code> ، <code>on_submit</code> ، <code>on_cancel</code> و <code>on_update_after_submit</code>
Karyar	تکمیل گام، تصمیم انسانی، پیشنهاد ایجنت، پیام کانال
بیرونی	Webhook ورودی، callback از n8n، نتیجه RPA، پاسخ مودیان
زمان	Scheduler، مثل «هر روز ساعت ۸»، و تایمرهای مهلت

۹.۲ Transactional Outbox

```
doc_event (same DB transaction as the business change)
└─ insert "Karyar Event" row {status: pending}
frappe.db.after_commit → enqueue("karyar.events.dispatch", queue="karyar_events")
dispatcher:
  match subscriptions (workflow triggers, waiting steps, outbound webhooks/n8n)
  → create/advance Workflow Runs (idempotent by event_id)
  → mark event dispatched
scheduler sweep (every minute): re-dispatch pending events older than N seconds
```

- چرا Outbox؟ اگر رویداد مستقیماً بعد از تغییر سند به صف فرستاده شود، ممکن است تراکنش rollback شود و رویداد «دروغ» منتشر شود، یا برعکس، commit انجام شود ولی صف از دست برود. Outbox هر دو را حل می‌کند: تحویل دست‌کم یک‌بار (at-least-once) همراه با مصرف‌کننده idempotent.
- پوشش Wildcard: استفاده از "*" در doc_events روی همه DocType ها پرهزینه است. فقط DocType هایی که در پیکربندی «Event Source» فعال شده‌اند (با کش) رویداد تولید می‌کنند.

۹.۳ پوشش استاندارد رویداد

```
{ "event_id": "uuid", "type": "erp.sales_order.submitted", "schema_version": 1,
  "tenant": "acme.karyar.ir", "occurred_at": "...",
  "subject": {"doctype": "Sales Order", "name": "S0-0042"},
  "actor": {"type": "human|agent|system|customer", "id": "..."},
  "correlation_id": "run/RUN-0007", "causation_id": "event/...",
  "payload": { minimal, non-sensitive fields } }
```

payload عمده حداقلی است. مصرف‌کننده برای جزئیات، داده را با ابزار و با مجوز خودش از ERPNext می‌خواند. این کار از نشت داده از طریق رویدادها جلوگیری می‌کند.

۹.۴ چرا Kafka یا RabbitMQ نه (فعلاً)؟

Redis Queue (RQ) و Outbox مبتنی بر MariaDB همین حالا در Frappe وجود دارند و برای بار یک Tenant کافی‌اند. وقتی به جریان رویداد بین Tenantها نیاز شد، مثلاً برای اندازه‌گیری مرکزی مصرف یا تحلیل، می‌شود یک **Relay** از Outbox به NATS، Redis Streams یا Kafka اضافه کرد. هیچ تولیدکننده‌ای لازم نیست تغییر کند.

۱۰. ارتباط ایجنت با ایجنت

اصل: ایجنت‌ها با هم «چت» نمی‌کنند. ارتباط فقط از سه راه است:

۱. تحویل از راه گردش کار (اصلی):

- ایجنت A گام خود را با **خروجی ساخت‌یافته** (طبق Schema) تمام می‌کند و خروجی در `context` پرونده ذخیره می‌شود.
 - رویداد `workflow.step.completed` منتشر می‌شود.
 - گام بعدی به نقش کاری X تعلق دارد. جدول انتساب می‌گوید X با ایجنت B است.
 - ایجنت B یک **دستور کار (Task Brief)** دریافت می‌کند که از `context` ساخته شده است، نه متن خام گفت‌وگوی A.
۲. **تحویل از راه ERPNext**: ایجنت A سندی می‌سازد (مثلاً Lead)، رویداد ERPNext منتشر می‌شود و گردش کار دیگری که trigger آن «Lead ایجاد شد» است برای ایجنت B شروع می‌شود.
۳. **مشاوره کنترل‌شده (آینده)**: ابزار `consult(responsibility, question)` که فقط اطلاعات **فقط‌خواندنی** و در محدوده مجوز برمی‌گرداند و ثبت می‌شود. در فاز ۱ وجود ندارد.

چرا Task Brief به جای متن گفت‌وگو؟

- جلوگیری از انتقال **Prompt Injection**: متن مشتری ممکن است دستور مخرب داشته باشد.
- کاهش هزینه `token`.
- کمینه‌سازی داده: ایجنت حسابداری لازم نیست کل گفت‌وگوی پزشکی را ببیند.
- قابلیت ممیزی: مشخص است دقیقاً چه چیزی تحویل شده است.

۱۱. معماری مجوز

۱۱.۱ زنجیره مجوز

Tenant (site)	→ which packs/capabilities/integrations are licensed & enabled
↳ Principal	→ who is acting: Human User Agent Service User Channel Contact Integration
↳ Agent	→ which Responsibilities → Capabilities it may use
↳ Step scope	→ which Capabilities & which records are allowed in THIS workflow step
↳ Frappe	→ DocType perms, User Permissions, permlevel, has_permission hooks
↳ Output filter	→ field whitelist per tool + masking by data classification
Effective = Tenant n Agent n Step n Principal(Frappe)	– evaluated in code, never in the prompt

۱۱.۲ اصل «از طرفِ چه کسی؟» (On-behalf-of)

موقعیت	هویت اجرایی	نتیجه
کارمند با ایجنت گفت‌وگو می‌کند	کاربر همان کارمند (<code>frappe.set_user</code>)، به‌علاوه فیلتر توانمندی ایجنت	ایجنت هرگز بیش از خود کارمند نمی‌بیند
گام خودکار گردش‌کار (بدون انسان)	کاربر سرویس ایجنت، با <code>Frappe Role</code> های مشتق از توانمندی‌ها، محدود به رکوردهای همین پرونده	دسترسی حداقلی
مشتری بیرونی در کانال	«Contact Principal» بدون دسترسی <code>Desk</code> . ابزارهای ویژه که فقط داده همان مخاطب را برمی‌گردانند	مشتری فقط داده خودش را می‌بیند
n8n یا سیستم بیرونی	کاربر یکپارچه‌سازی با توانمندی‌های تعریف‌شده و کلید <code>API</code> یا <code>OAuth</code>	محدود به <code>Karyar API</code>

مثال سؤال «سود کل شرکت چقدر است؟» از کارمند عادی:

1. ایجنت ممکن است ابزار `report.profit_and_loss` را صدا بزند.
 2. `Tool Executor` بررسی می‌کند: آیا این کارمند به گزارش «Profit and Loss Statement» دسترسی `Frappe` دارد؟ جواب منفی است.
 3. نتیجه `PERMISSION_DENIED` برمی‌گردد و هیچ داده‌ای به مدل نمی‌رسد.
 4. ایجنت فقط توضیح می‌دهد که دسترسی ندارد.
- حتی اگر `prompt` دستکاری شود، داده‌ای برای نشت وجود ندارد.

۱۱.۳ قواعد پیاده‌سازی (قابل بررسی خودکار)

- ابزارها فقط از `API` های مجوزدار استفاده می‌کنند: `doc.check_permission` ، `frappe.get_list` ، `frappe.has_permission` ، `query_report.run` .
- استفاده از `frappe.get_all` ، `ignore_permissions=True` و `frappe.db.sql` در مازول ابزارها ممنوع است. این قاعده با یک قانون `semgrep` در `CI` بررسی می‌شود (`ERPNext` خودش `semgrep` دارد). استثنا فقط با بازبینی امنیتی و توضیح مکتوب مجاز است.
- هر ابزار یک **Schema خروجی** دارد. فیلدهای خارج از `Schema` حذف می‌شوند. فیلدهای دارای `permlevel` بالاتر از سطح دسترسی `principal` پوشانده می‌شوند.
- طبقه‌بندی داده** (فاز ۲): فیلدها با برچسب‌هایی مثل `pii` ، `financial` ، `medical` علامت‌گذاری می‌شوند. سیاست `Tenant` تعیین می‌کند کدام برچسب‌ها اجازه دارند به ارائه‌دهنده بیرونی `AI` فرستاده شوند (بخش ۱۹).

۱۱.۴ محدودیت `Frappe` که باید صریح گفته شود

- مدل مجوز `Frappe` نقش‌محور (**RBAC**) به‌علاوه **User Permission** است، نه **ABAC** کامل. منطق ظریف‌تر از راه `hook` های `has_permission` و `permission_query_conditions` پیاده می‌شود.
- گزارش‌های **Query/Script** ممکن است فقط در سطح نقش کنترل شوند و `User Permission` را کامل اعمال نکنند. بنابراین ایجنت فقط به فهرست گزینش‌شده گزارش‌ها (`Report Catalog`) دسترسی دارد. هر گزارش قبل از ورود به فهرست، از نظر مجوز بازبینی می‌شود.

۱۲. معماری یکپارچگی با ERPNext

موضوع	تصمیم
خواندن	از ابزارها، با <code>get_list</code> یا <code>get_doc</code> و مجوز <code>principal</code>
نوشتن	فقط از ابزارهای <code>draft-write</code> (سند در وضعیت <code>docstatus=0</code>) و <code>commit-write</code> (یا <code>submit</code> یا <code>cancel</code>)
منطق کسب و کار	از کنترلرهای ERPNext استفاده می‌شود (<code>doc.insert()</code> ، <code>doc.submit()</code> ، توابع <code>make_*</code> مثل <code>make_sales_invoice</code>)، نه نوشتن مستقیم جدول. همه اعتبارسنجی‌های ERPNext حفظ می‌شوند
فیلدهای اضافه	<code>Custom Field</code> و <code>Property Setter</code> به صورت <code>fixtures</code> در اپ <code>Karyar</code> یا <code>Pack</code> . هرگز تغییر <code>DocType</code> JSON‌های ERPNext
رفتار اضافه	<code>doc_events</code> و <code>extend_doctype_class</code> از <code>override_doctype_class</code> فقط در صورت اجبار و با مستندسازی استفاده می‌شود. Monkey-patch ممنوع است
وضعیت فرایند	در <code>Karyar Run</code> نگه داشته می‌شود. وضعیت سند (<code>docstatus</code> و <code>status</code>) در ERPNext است. Frappe Workflow روی <code>DocType</code> هایی که <code>Karyar</code> مدیریت می‌کند فعال نمی‌شود تا دو ماشین حالت روی یک سند نداشته باشیم (بخش ۳۳)
ارجاع	<code>Karyar Run</code> فقط ارجاع (<code>name / doctype</code>) به سند ERPNext نگه می‌دارد، نه کپی داده. تنها استثنا <code>snapshot</code> لازم برای ممیزی تأیید است
Idempotency	فیلد سفارشی <code>karyar_idempotency_key</code> روی <code>DocType</code> هایی که ابزارها می‌سازند، همراه با بررسی یکتایی

نمایه ابزار (Tool Contract):

```
@karyar_tool(  
    name="crm.lead.create_draft", capability="crm.lead.draft",  
    side_effect="draft-write", risk="low",  
    input_schema=LeadDraftIn, output_schema=LeadRef,  
    required_perms=[("Lead", "create")], idempotent=True)  
def create_lead_draft(inp: LeadDraftIn, ctx: ToolContext) -> LeadRef: ...
```

۱۳. معماری Karyar API

- یک لایه سرویس، سه نمایش. منطق در «سرویس‌های توانمندی» نوشته می‌شود و از سه راه در دسترس قرار می‌گیرد:
 1. ابزارهای ایجنت
 2. REST: `/api/method/karyar.api.v1.<resource>.<action>` یا مسیرهای v2 در Frappe
 3. (فاز بعد) MCP با کتابخانه `frappe-mcp` برای ایجنت‌های بیرونی
- نسخه‌داری: v1 پایدار است. تغییرات فقط افزایشی‌اند و تغییر ناسازگار یعنی v2. DTOها با Schema صریح تعریف می‌شوند، نه ساختار خام `DocType`. به این ترتیب تغییر نام فیلد در ERPNext نسخه ۱۷ مصرف‌کننده‌ها را نمی‌شکند.
- احراز هویت:
 - کلید API/Secret برای سیستم‌ها (n8n)
 - OAuth2 برای اپ‌ها و ایجنت‌های بیرونی (Frappe خودش OAuth Provider است)
 - Session برای Workspace
- محدودیت نرخ و اندازه برای هر `principal`.

- منابع اولیه v1 (حداقلی): messages ، conversations (ورودی کانال‌ها)، runs (شروع و وضعیت پرونده)، tasks (صندوق کار انسانی)، events/callback (callback از n8n و RPA)، و tools/invoke (فقط برای principal‌های مجاز).
- Webhook خروجی: با سازوکار Webhook در Frappe و امضای HMAC، یا از دیسپچر رویداد Karyar.

۱۴. معماری گزارش‌گیری

اولویت	منبع	سازوکار
۱	گزارش‌های استاندارد ERPNext	ورود به Report Catalog با شناسه، توضیح، Schema فیلترهای مجاز و ستون‌های مجاز. اجرا با <code>frappe.desk.query_report.run</code> با هویت principal
۲	گزارش‌های Karyar	تابع Python از پیش نوشته‌شده با Schema ورودی و خروجی که در کد بازبینی شده است. ترکیبی‌ها هم همین‌جا هستند، مثل «خلاصه مدیریتی = فروش + دریافتنی + موجودی بحرانی»
۳ (بعد)	Frappe Insights	برای داشبوردهای انسانی، نه برای پرسش آزاد ایجنت
×	SQL ساخته‌شده توسط هوش مصنوعی	ممنوع

نقش هوش مصنوعی در گزارش:

۱. فهم درخواست
 ۲. انتخاب گزارش از فهرست مجاز
 ۳. پر کردن پارامترها (با اعتبارسنجی Schema)
 ۴. تفسیر و توضیح
- اعداد همیشه از نتیجه گزارش می‌آیند. مدل محاسبه نمی‌کند. اگر نسبت یا جمع لازم است، گزارش Karyar آن را محاسبه می‌کند. پاسخ ایجنت منبع را هم نشان می‌دهد: نام گزارش، فیلترها و زمان اجرا.
- (فاز ۲) بررسی عدد: یک بررسی خودکار کنترل می‌کند هر عدد در متن پاسخ، در داده نتیجه وجود داشته باشد.
- عملکرد: گزارش‌های سنگین به صورت Prepared Report اجرا می‌شوند. در صورت نیاز از Read Replica پایگاه داده استفاده می‌شود که Frappe پشتیبانی می‌کند.

۱۵. معماری مدل هوش مصنوعی

```
Agent Runtime → AIGateway.complete(ModelRequest) → ProviderAdapter → Provider/Model
|
| alias → (provider, model, params, fallback chain) [per tenant]
| structured-output enforcement (JSON Schema validate + 1 retry)
| redaction hook (data classification policy)
| timeouts / retries / circuit breaker
└─ UsageSink.record(UsageRecord) ← extension point (section 16)
```

• آداپتورها:

- «سازگار با OpenAI»: بیشتر ارائه‌دهندگان، درگاه‌های داخلی و مدل‌های خودمیزبان (vLLM و Ollama) را پوشش می‌دهد.
- آداپتورهای اختصاصی، مثلاً Anthropic و Google، در صورت نیاز.

◦ همه آداپتورها از راه `karyar_ai_providers hook` قابل افزودن اند.

- **پیکربندی مدل:**

- `Model Profile` در هر `Tenant`، نام مستعار (`fast`، `smart`، `local-private`) را به مدل واقعی نگاشت می‌کند.
- ایجنت فقط نام مستعار را می‌شناسد. تعویض ارائه‌دهنده یعنی تغییر یک رکورد.

- **دستورالعمل‌ها:** `Prompt Template` نسخه‌دار. هر نوبت ایجنت نسخه دستورالعمل را ثبت می‌کند تا بشود رفتار را بازتولید کرد.

- **کلیدها:** «کلید سکو» (`Karyar` هزینه `AI` را می‌فروشد) یا «کلید مشتری» (`BYOK`). هر دو در فیلد `Password` سایت ذخیره می‌شوند. **تصمیم تجاری لازم است.**

- **LiteLLM Proxy:** گزینه فاز ۲ یا ۳، وقتی بودجه‌بندی، کلیدهای مجازی و مسیریابی مرکزی لازم شد. چون همه ترافیک از `AI Gateway` می‌گذرد، جایگزینی بدون تغییر ایجنت‌هاست.

- **⚠️ واقعیت ایران:**

- دسترسی به برخی ارائه‌دهندگان بین‌المللی از ایران به دلیل تحریم و سیاست ارائه‌دهندگان محدود است.
- فرستادن داده مشتری (به‌ویژه داده پزشکی یا مالی) به خارج پیامد حقوقی دارد.
- معماری باید از روز اول **مدل خودمیزبان یا داخلی** را به‌عنوان گزینه درجه‌یک پشتیبانی کند.
- کیفیت زبان فارسی هر مدل باید با **مجموعه ارزیابی فارسی** سنجیده شود.

۱۶. نقطه توسعه هزینه و (AI Cost / Token Tracking) token

الان ساخته نمی‌شود؛ فقط جای تمیز آن گذاشته می‌شود:

UsageRecord در هر فراخوانی **AI Gateway**:

```
request_id, tenant, agent, responsibility, workflow, run, step, conversation, api_principal,
provider, model, model_alias, input_tokens, output_tokens, cached_tokens, total_tokens,
latency_ms, status, price_version, estimated_cost, currency, actual_cost (nullable), timestamp
```

- **UsageSink**: یک رابط است. در فاز ۱ دو پیاده‌سازی دارد:

1. `Karyar AI Usage DocType` در همان سایت

2. یک خط لاگ `JSON` ساخت‌یافته

بعداً پیاده‌سازی «ارسال به سرویس اندازه‌گیری مرکزی» در `Control Plane` اضافه می‌شود، که برای جمع بین `Tenant`‌ها لازم است چون هر `Tenant` پایگاه داده جدا دارد. - **قیمت‌ها:** جدول `Model Price` نسخه‌دار با تاریخ اعتبار. **در کد ثابت نیستند.** `actual_cost` بعداً با صورت‌حساب ارائه‌دهنده تطبیق داده می‌شود. - **حداقل لازم از روز اول:** یک **سقف سخت** مصرف روزانه برای هر `Tenant` و هر ایجنت (قطع‌کننده مدار). یک حلقه معیوب ایجنت می‌تواند در چند ساعت هزینه سنگین بسازد.

۱۷. معماری n8n

کجا؟	چه چیزی؟
ERPNext	داده و قواعد کسب‌وکار: سند، حسابداری، انبار، اعتبارسنجی‌ها، وضعیت سند، مجوز پایه

کجا؟	چه چیزی؟
Karyar	ایچنت‌ها، گردش کارهای کسب و کار، HITL، رویداد، سیاست، ابزار، API، ممیزی، AI Gateway، و هر تصمیم یا تراکنش حساس
n8n	اتصال به سرویس‌های ثالث (پیامک، ایمیل مارکتینگ، CRM، Google Sheets، دیگر)، اعلان‌ها، همگام‌سازی‌های زمان‌بندی‌شده، خروجی داده، چسب‌های کم‌ریسک

قواعد سخت:

1. n8n هرگز مستقیم در ERPNext نمی‌نویسد. فقط از **Karyar API** و با کاربر یکپارچه‌سازی و توانمندی‌های محدود کار می‌کند.

2. n8n وضعیت کسب و کار نگه نمی‌دارد. اگر چیزی باید یادش بماند، جای آن در Karyar یا ERPNext است.

3. Karyar از گام integration_call با یک callback امضا شده، n8n را صدا می‌زند و پرونده تا رسیدن callback یا timeout منتظر می‌ماند.

4. callback‌های حساس (مثل تأیید پرداخت درگاه بانکی) مستقیم به **Karyar API** می‌آیند، نه از مسیر n8n.

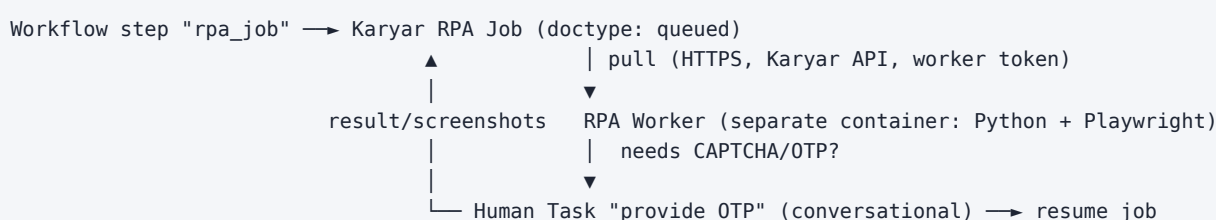
5. workflow‌های n8n به صورت JSON در مخزن karyar-deploy نسخه‌داری می‌شوند.

چند مستأجری: n8n نسخه Community چندمستأجری واقعی ندارد. پیشنهاد:

- یک n8n مشترک برای جریان‌های سکو که پارامتر Tenant می‌گیرند و اعتبارنامه‌ها را از Karyar می‌گیرند.
- n8n اختصاصی فقط برای مشتریانی که جریان سفارشی دارند.

⚠ **مجوز:** n8n مجوز Sustainable Use دارد. اگر مشتریان شما با اعتبارنامه‌های خودشان از n8n (حتی غیرمستقیم) استفاده کنند، احتمالاً مجوز Embed لازم است. قبل از تعهد تجاری بررسی حقوقی شود. جایگزین‌های متن‌باز: Activepieces (MIT در هسته) یا Node-RED (Apache-2.0).

۱۸. معماری RPA



اصل	توضیح
اول API	RPA فقط برای سیستم‌هایی است که API ندارند، مثل پورتال بانک، تأمین اجتماعی، سامانه‌های دولتی و نرم‌افزارهای قدیمی
هرگز روی رابط ERPNext	برای کار داخل ERPNext همیشه ابزار و API استفاده می‌شود
سرویس جدا	Worker در کانتینر ایزوله و بدون دسترسی به پایگاه داده اجرا می‌شود. فقط با Karyar API ارتباط دارد
مدل Pull	Worker خودش کار را برمی‌دارد. لازم نیست ورودی شبکه به Worker باز باشد
اسرار	اعتبارنامه پورتال در فیلد Password سایت Tenant ذخیره می‌شود و فقط برای همان Job و برای مدت کوتاه به Worker داده می‌شود

توضیح	اصل
کپچا، OTP و تأیید دستی تبدیل به Human Task می‌شوند. کارمند به صورت گفت‌وگویی کد را وارد می‌کند و Job ادامه پیدا می‌کند	انسان در حلقه
اسکرین‌شات و لاگ هر گام به صورت فایل خصوصی پیوست Job می‌شود	شواهد
فاز ۳ به بعد. الان لازم نیست	زمان

۱۹. معماری چندمستأجری (Multi-Tenant)

۱۹.۱ گزینه‌ها

مدل	جداسازی	هزینه و عملیات	ارتقا	مناسب برای
A. یک Site، چند Company در ERPNext	ضعیف. با یک خطای User Permission، داده نشت می‌کند. داده پایه مثل Item و Customer مشترک است. سفارشی‌سازی‌ها مشترک‌اند	ارزان‌ترین	با هم	فقط یک گروه تجاری با چند شخصیت حقوقی. ❌ برای کسب‌وکارهای مستقل
B. یک Site برای هر Tenant روی Bench مشترک ✓	پایگاه داده جدا با کاربر جدا، پوشه فایل جدا، کلید رمزنگاری جدا. پروسه‌ها، Workerها و Redis مشترک‌اند (کلیدها با نام سایت جدا می‌شوند)	متوسط. هر Bench می‌تواند ده‌ها سایت داشته باشد	همه سایت‌های یک Bench با هم ارتقا می‌یابند	پیش‌فرض Karyar. همان مدلی که Frappe Cloud استفاده می‌کند
C. یک Bench یا Stack برای هر Tenant	قوی‌ترین: کانتینر، Worker و حتی سرور پایگاه داده جدا	گران و پر عملیات	مستقل	مشتریان بزرگ، حساس (پزشکی) یا نیازمند نسخه خاص
D. Row-Level Security در پایگاه داده	—	—	—	در ERPNext و Frappe قابل استفاده نیست

۱۹.۲ پیشنهاد

- پیش‌فرض: مدل B با «گروه‌های Bench» (مثلاً bench-standard-01 و bench-standard-02).
- برای مشتریان حساس: مدل C.
- مدل A برای کسب‌وکارهای جدا هرگز استفاده نشود.

نکات صریح:

- «جدول جدا» یا حتی «پایگاه داده جدا» به تنهایی کافی نیست. در مدل B، کد همه Tenantها در یک پروسه اجرا می‌شود. پس:
 - هیچ کد مخصوص یک مشتری بدون بازبینی روی Bench مشترک نصب نمی‌شود.
 - Server Script و دسترسی bench، Administrator و console به مشتری داده نمی‌شود.
 - Workerها و کش Redis همیشه با frappe.local.site کار می‌کنند.
- همه اجزای بیرون از Frappe باید Tenant-aware باشند. n8n، RPA، AI Gateway (در صورت جدا شدن) و Channel Gateway (در صورت جدا شدن) هر درخواست را با شناسه سایت و اعتبارنامه همان سایت انجام می‌دهند.

- **محدودیت مقیاس مدل B: Scheduler** روی همه سایت‌ها می‌چرخد و Workerها مشترک‌اند، پس هر Bench سقفی دارد. مشتری پرمصرف به Bench دیگری منتقل می‌شود (پشتیبان‌گیری و بازیابی سایت).
- **Control Plane (بعد):** شامل ثبت Tenantها، Bench هر Tenant، Packها، پلن، سقف مصرف و جمع مصرف. می‌تواند یک سایت Frappe جداگانه با اپ `karyar_platform` باشد. Frappe Press (AGPL) را می‌توان بررسی کرد، ولی برای شروع سنگین است. اول با اسکریپت راه‌اندازی شود: `new-site`، سپس `install-app`، سپس `apply-pack`.

۲۰. معماری امنیت

تصمیم	حوزه
مدل B یا C (بخش ۱۹). هر سایت کلید رمزنگاری خودش را دارد (<code>encryption_key</code>)	جداسازی Tenant
ورود Frappe همراه با 2FA (موجود در Frappe) برای نقش‌های مالی و مدیریتی. برای کانال‌ها: اتصال حساب کانال به کاربر از راه کد یک‌بارمصرف	احراز هویت انسان
هویت کانال (شناسه تلگرام یا شماره واتس‌اپ). برای دسترسی به داده حساس، تأیید با OTP پیامکی	احراز هویت مشتری
API Key/Secret یا OAuth2 برای هر سیستم بیرونی، با چرخش کلید	احراز هویت ماشین
بخش ۱۱: زنجیره کامل در کد و ممنوعیت <code>ignore_permissions</code> در ابزارها	مجوز
فیلدهای Password در Frappe (رمزنگاری با کلید سایت). کلیدهای سکو در متغیر محیطی یا Vault. هرگز در <code>prompt</code> ، لاگ یا Pack	اسرار
تأیید امضای HMAC روی همه Webhookها، جلوگیری از replay با <code>timestamp</code> و <code>nonce</code> ، محدودیت نرخ بیرونی	ورودی‌های بیرونی
محتوای مشتری، فایل‌ها و صفحات وب «داده نامطمئن» برچسب می‌خورند. اختیار اقدام فقط از <code>principal</code> و سیاست می‌آید. ابزارهای پریسک بدون HITL در دسترس گفت‌وگوی مشتری نیستند	Prompt Injection
گره خوردن تأیید به <code>payload_hash</code> ، احراز هویت تأییدکننده، و تأیید قوی برای عملیات مالی (بخش ۸)	امنیت تأیید
Schema خروجی ابزار، طبقه‌بندی داده، و امکان «مدل خودمیزبان» برای داده حساس	کمینه‌سازی داده به AI
ایزوله، بدون دسترسی به پایگاه داده، اعتبارنامه کوتاه‌مدت	RPA
Audit Log فقط افزودنی (بخش ۲۱)	ممیزی
پین کردن نسخه‌ها، mirror داخلی PyPI و npm (به دلیل تحریم)، اسکن وابستگی‌ها	زنجیره تأمین

۲۱. معماری ممیزی و مشاهده‌پذیری

۲۱.۱ سه لایه ردیابی

لایه	محتوا	ابزار
تغییرات داده	چه فیلدی در کدام سند تغییر کرد	Version در Frappe (track changes) — موجود

لايه	محتوا	ابزار
ممیزی معنایی	چه کسی (انسان، ایجنت، سیستم یا مشتری)، از طرف چه کسی، کدام توانمندی یا ابزار، ورودی (خلاصه یا hash)، پیشنهاد در برابر اجرای واقعی، تأییدکننده، سند اثرپذیرفته، پرونده و گام، نتیجه و خطا	Karyar Audit Log جدید DocType
ردیابی فنی	زمان‌ها، retry، مصرف AI، خطاها	RQ و Step Run، AI Usage، Error Log Job، با correlation_id مشترک

- فقط افزودنی: هیچ نقشی، حتی System Manager، اجازه ویرایش یا حذف Audit Log را ندارد. این در کنترلر اعمال می‌شود.
- محدودیت صریح: مدیر پایگاه داده همچنان می‌تواند با SQL تغییر دهد. برای عملیات حساس در فاز بعد، زنجیره hash (هر ردیف hash ردیف قبل را دارد) برای تشخیص دستکاری اضافه می‌شود.
- ذخیره prompt و پاسخ مدل: با سطح‌های قابل تنظیم off، summary و full، به‌علاوه سیاست نگهداری. این توازن بین عیب‌یابی و حریم خصوصی است.

۲۱.۲ مشاهده پذیری

الان:

- correlation_id در همه لاگ‌ها
- لاگ ساخت‌یافته JSON
- یک داشبورد ساده در Workspace: پرونده‌های گیرکرده، کارهای انسانی معوق، خطاها، صف‌ها، و مصرف AI امروز

بعد:

- OpenTelemetry برای trace بین n8n، Worker، Frappe و RPA
- Prometheus و Grafana برای متریک‌ها
- Sentry یا GlitchTip برای خطاها
- هشدارها

۲۲. جریان داده

```
[Customer msg] → Channel Gateway → Conversation(+Message) → event channel.message.received
→ Workflow Engine: find/create Run → agent_task(Ava, responsibility=intake)
→ Ava: tools (validate_phone, validate_national_id, ...) → Run.context.intake (draft, NOT ERP yet)
→ customer confirmation (hash h1) → human_task(review) → decision approve(h2)
→ erp_action: crm.lead.create (idempotent) → ERPNext Lead (SOURCE OF TRUTH)
→ doc_event → Karyar Event erp.lead.created (outbox)
→ trigger workflow "sales_followup" → agent_task(responsibility=sales → Hanna)
```

مرز داده:

- Karyar فقط وضعیت فرایند را نگه می‌دارد: گفت‌وگو، context پرونده، پیش‌نویس‌های قبل از تأیید، تصمیم‌ها و ممیزی.
- بعد از تأیید، حقیقت کسب‌وکار وارد ERPNext می‌شود و Karyar فقط به آن ارجاع می‌دهد.
- اگر کسی Lead را مستقیم در ERPNext تغییر دهد، ERPNext معتبر است. ایجنت‌ها همیشه داده تازه را از ERPNext می‌خوانند، نه از context کهنه پرونده.

۲۳. ساختار مخزن‌ها

۲۳.۱ تعارض فعلی

مخزن `karyar-mainERP` خودِ ERPNext است و روی `develop` قرار دارد. اگر `Karyar` داخل آن نوشته شود:

- (۱) عملاً هسته تغییر کرده است.
- (۲) هر ادغام با `upstream` تعارض می‌دهد.
- (۳) `bench` اپ‌ها را یک مخزن برای هر اپ نصب می‌کند.

۲۳.۲ پیشنهاد: چند مخزن (Polyrepo) و یک مخزن استقرار

مخزن	محتوا	فاز
<code>karyar-mainERP</code> (همین)	ERPNext، با پایه <code>version-16</code> (شاخه <code>karyar/version-16</code>). فقط وصله‌های اجتناب‌ناپذیر، همراه با فهرست <code>PATCHES.md</code> و تلاش برای ارسال به <code>upstream</code>	۰
<code>karyar</code>	اپ اصلی سکو (ساختار پایین)	۱
<code>karyar_iran</code>	بومی‌سازی ایران و مودیان	۲
<code>karyar_hr_ir</code>	حقوق، بیمه و مالیات حقوق ایران (روی HRMS)	۳ به بعد
<code>karyar-deploy</code>	<code>apps.json</code> با نسخه‌های پین‌شده، <code>Dockerfile</code> و <code>compose</code> (بعدها Helm)، اسکریپت راه‌اندازی <code>workflow</code> ، <code>Tenant</code> ‌های <code>Pack</code> ، <code>n8n</code> ، <code>runbook</code> ‌ها	۰ تا ۱
<code>karyar-rpa-worker</code>	سرویس RPA	۳ به بعد
<code>karyar-packs</code> (اختیاری)	اگر <code>Pack</code> ‌ها زیاد شدند، به‌صورت اپ یا مخزن جدا	بعد

چرا اپ `karyar` یکی است، نه چند اپ از روز اول؟

- جدا کردن اپ‌ها هزینه نسخه‌داری و وابستگی دارد.
- مرز را با ماژول‌ها و پکیج‌های `Python` و قاعده `import` می‌کشیم.
- ریسک: انتقال `DocType` بین اپ‌ها در آینده دردسر دارد. پس مرزهای ماژول باید از اول تمیز باشند.

```

karyar/                                     (Frappe app repo)
├─ pyproject.toml
├─ karyar/
│   ├── hooks.py                           # doc_events, scheduler, extension-point hook names, fixtures
│   ├── modules.txt                        # Karyar Agents, Karyar Workflow, Karyar Conversations,
│   │                                     # Karyar Events, Karyar Integrations, Karyar Audit, Karyar AI
│   │                                     # framework-light domain logic (unit-testable)
│   ├── core/
│   │   ├── workflow/ (graph model, executor, step types registry)
│   │   ├── policy/   (permission chain, risk gates)
│   │   ├── tools/    (registry, executor, contracts)
│   │   ├── agents/   (runtime loop, context builder)
│   │   ├── ai/       (gateway, provider adapters, usage sink)
│   │   └─ events/    (envelope, outbox, dispatcher)
│   ├── karyar_agents/doctype/            # Karyar Agent, Responsibility, Capability, Role Assignment,
│   │                                     # Prompt Template, Model Profile
│   ├── karyar_workflow/doctype/          # Workflow Definition, Workflow Run, Step Run, Human Task, Timer
│   ├── karyar_conversations/doctype/      # Conversation, Message, Channel Account, Contact Identity
│   ├── karyar_events/doctype/            # Karyar Event, Event Source, Event Subscription
│   ├── karyar_integrations/doctype/       # Integration Endpoint, RPA Job
│   ├── karyar_audit/doctype/             # Audit Log, AI Usage, Model Price
│   ├── tools/                           # built-in tool packs: crm, selling, accounts, stock, reports, task
│   ├── channels/                         # adapters: web widget, telegram, (bale via karyar_iran)
│   ├── api/v1/                           # stable REST facade
│   ├── mcp.py                            # (later) frappe-mcp exposure of the same tools
│   ├── packs/                            # base packs (fixtures/JSON)
│   └─ workspace/                         # Karyar Workspace SPA (React + frappe-react-sdk, like
erpnext/banking)
│   ├── commands/                         # bench karyar apply-pack, etc.
│   ├── patches/ , patches.txt
│   ├── locale/fa.po
│   └─ tests/                             # unit + integration (against ERPNext) + tool contract tests

```

الگوی موجود: ERPNext خودش یک SPA با React برای بانکداری دارد که با `frappe-react-sdk` در `/banking` سرو می‌شود. Karyar Workspace می‌تواند دقیقاً همین الگو را دنبال کند، یعنی بدون سرور جداگانه فرانت‌اند. گزینه دیگر **frappe-ui** است که اپ‌های CRM و Helpdesk استفاده می‌کنند. (تصمیم لازم است).

۲۴. معماری استقرار

فاز ۱ (یک سرور یا Docker Compose، بر پایه `frappe_docker`):

```

nginx (TLS, per-tenant hostnames)
├─ frappe-web (gunicorn)           └─ socketio (realtime)
├─ workers: short, default, long, karyar_events, karyar_agent (concurrency-limited)
├─ scheduler
├─ MariaDB (one DB per site)       └─ redis-cache └─ redis-queue
├─ n8n (+ its own Postgres)        [phase 2]
└─ backups → S3-compatible object storage (encrypted)

```

فاز ۲ و بعد:

- چند Kubernetes Bench، Helm chart رسمی Frappe وجود دارد
- replica با MariaDB
- RPA Worker ها
- (در صورت نیاز) سرویس جدای Channel Gateway و AI Gateway (LiteLLM)

⚠️ **تصمیم محل میزبانی (بزرگ‌ترین تصمیم زیرساختی):**

گزینه	مزیت	عیب
داخل ایران	اقامت داده، دسترسی مشتریان، مودیان و بانک‌ها	دسترسی به ارائه‌دهندگان AI بین‌المللی و APIهای تلگرام و Meta محدود است
خارج از ایران	دسترسی به AI و کانال‌های بین‌المللی	ریسک حقوقی و اقامت داده، و دسترسی سامانه‌های داخلی
ترکیبی (پیشنهاد اولیه)	ERP و داده در ایران. یک «Egress Gateway» کم‌حجم بیرون، فقط برای AI و کانال‌های بین‌المللی، همراه با کمینه‌سازی داده	پیچیدگی بیشتر و نیاز به بررسی حقوقی

۲۵. راهبرد مقیاس‌پذیری

- **گلوگاه اصلی تأخیر و نرخ API مدل است، نه پایگاه داده. پس:**
 - صف جدا برای ایجنت‌ها
 - **سقف همزمانی برای هر Tenant** (عدالت بین Tenantها)
 - پخش پاسخ (streaming)
 - کش پاسخ ابزارهای خواندنی پرتکرار برای مدت کوتاه
- **افقی:** Worker و gunicorn بیشتر. Workerهای karyar_agent مستقل از Workerهای ERPNext مقیاس می‌گیرند.
- **عمودی پایگاه داده:** MariaDB قوی‌تر، replica برای گزارش‌ها، و Prepared Report.
- **بین Tenantها:** انتقال Tenant پرمصرف به Bench جدید و سقف تعداد سایت در هر Bench.
- **رویداد:** Outbox و RQ تا هزاران رویداد در دقیقه برای هر Bench کافی است. بعد از آن Relay به Streams یا NATS.
- **قبل از استخراج سرویس:** ابتدا اندازه‌گیری و پروفایل‌گیری. سرویس جدا فقط وقتی ساخته می‌شود که داده نشان دهد لازم است.

۲۶. راهبرد ارتقا

1. **ERPNext و Frappe دست‌نخورده یا تقریباً دست‌نخورده،** با نسخه‌های پین‌شده در `apps.json`. ارتقاها minor نسخه v16 به صورت ماهانه انجام می‌شوند: اول staging، بعد production.
2. **آزمون قرارداد ابزارها (Tool Contract Tests):** هر ابزار روی نسخه هدف ERPNext تست می‌شود. شکست در این تست‌ها یعنی ارتقا متوقف می‌شود.
3. **Karyar API v1** مصرف‌کننده‌ها را از تغییر نام‌ها در ERPNext محافظت می‌کند.
4. **نسخه‌داری** تعریف گردش کار، Pack و دستورالعمل‌ها. پرونده‌های در جریان روی نسخه خود باقی می‌مانند.
5. **ارتقای اصلی (v16 به v17):** حدود ۶ ماه بعد از انتشار پایدار v17، با شاخه آزمایشی و اجرای کل تست‌ها. احتمالاً همزمان فرصت ارزیابی PostgreSQL هم پیش می‌آید.
6. **بدون Monkey-patch.** درس از اپ‌های جامعه ایرانی: وصله‌های زمان اجرا بزرگ‌ترین مانع ارتقا هستند.

۲۷. معماری بومی سازی ایران

اپ karyar_iran (بدون هیچ وابستگی از سمت هسته karyar به آن):

بخش	طراحی	فاز
زبان و RTL	ترجمه (۹۶٪) Frappe و (۸۱٪) ERPNext موجود است. تکمیل ترجمه های Karyar در fa.po	۱
نرمال سازی متن	ارقام فارسی و عربی به لاتین، «ی/ك» به «ی/ك»، نیم فاصله، شماره موبایل (+98 / 09). به صورت ابزار قطعی که ایچنت ها و جست و جو استفاده می کنند	۱
اعتبارسنجی ها	کد ملی، شناسه ملی، کد اقتصادی، شبا (چک سام)، کد پستی. به صورت ابزار برای اعتبارسنجی Ava	۱
تاریخ جلالی	ذخیره میلادی و نمایش جلالی (datepicker, formatter, Jinja). تفسیر تاریخ های زبان طبیعی مثل «پنجشنبه هفته بعد» یا «۱۵ مهر» با یک ابزار قطعی parse_persian_date، نه حدس مدل. شماره گذاری اسناد با naming_series_variables	۱ تا ۲
کانال های ایرانی	آداپتور به (API شبیه تلگرام)، و در صورت نیاز ایتا یا روبیکا، از راه karyar_channel_adapters	۲
حسابداری	چارت حساب ایرانی با create_charts(custom_chart=...) . سطح «تفصیلی» با Party و Accounting Dimension. ریال با دقت صفر	۲
مالیات	ارزش افزوده قابل پیگیری (نه ثابت در کد)، کالاهای معاف، کسر از قرارداد (Tax Withholding Category)	۲
مودیان	ماژول جدا با کلاینت مستقل، صف ارسال، استعلام وضعیت، و HITL برای خطاها	۳
اسناد فارسی	قالب های چاپ RTL فاکتور، پیش فاکتور و رسید	۲
حقوق و بیمه	اپ karyar_hr_ir روی HRMS	۳ به بعد

بین المللی سازی بیش از حد ممنوع است. فقط دو کار از الان انجام می شود:

۱. هسته karyar به ایران وابسته نباشد.

۲. متن ها قابل ترجمه باشند.

چیزهایی مثل چندانری پیچیده یا قواعد مالیاتی چندکشوری الان طراحی نمی شوند.

۲۸. ریسک ها و توازن ها

#	ریسک	شدت	کاهش
R1	دسترسی به ارائه دهندگان AI از ایران، و حقوقی بودن ارسال داده به خارج	بالا	AI Gateway چندارائه دهنده ای، مدل خودمیزبان، طبقه بندی داده، بررسی حقوقی
R2	کانال های تلگرام، واتساپ و اینستاگرام در ایران فیلتر هستند و API رسمی Meta برای کسب و کار ایرانی در دسترس نیست	بالا	شروع با ویجت وب و به. Channel Gateway جدا شده تا کانال جایگزین شود
R3	ساختن موتور گردش کار، کار سختی است و می تواند دامنه پروژه را منفجر کند	بالا	مجموعه گام های حداقلی، تست های جدی، جدا کردن تعریف از اجرا کننده (مسیر فرار به Temporal)
R4	برداشت اشتباه مدل در تأیید گفت و گو محور	بالا	تصمیم ساخت یافته، تأیید صریح بر اساس ریسک، payload_hash

#	ریسک	شدت	کاهش
R5	Prompt Injection از پیام مشتری یا فایل	بالا	مجوز مستقل از مدل، داده نامطمئن، Task Brief به جای متن خام
R6	خلاً مجوز در گزارش‌های SQL خام ERPNext	متوسط	فقط Report Catalog بازبینی‌شده
R7	هزینه AI از کنترل خارج شود (حلقه ایجنت)	متوسط	سقف سخت از روز اول و محدودیت گام در هر نوبت
R8	پیکربندی بیش از حد، محصول را برای مشتری پیچیده کند	متوسط	Pack‌های آماده با پیش‌فرض‌های معقول. سازنده گرافیکی بعداً
R9	ارتقای ERPNext ابزارها را بشکند	متوسط	آزمون قرارداد، API facade، پین نسخه
R10	مجوزها: GPL و AGPL برای توزیع، و n8n برای SUL	متوسط	تصمیم مجوز Karyar و بررسی حقوقی
R11	داده حساس پزشکی (مثال کلینیک)	بالا (برای آن بخش بازار)	مدل C برای Tenant، مدل خودمیزبان، پوشاندن فیلدها
R12	تخصص کم تیم در Frappe	متوسط	شروع کوچک، الگوبرداری از کد ERPNext، آموزش

۲۹. تصمیم‌هایی که نیاز به تأیید دارند

۱. پایه: ERPNext و Frappe v16 به جای develop، و جابه‌جایی fork به version-16.
۲. ساختار مخزن: چند مخزن (karyar، karyar_iran، karyar-deploy).
۳. چندمستأجری: مدل B به عنوان پیش‌فرض و C برای حساس‌ها.
۴. محل میزبانی و سیاست ارسال داده به ارائه‌دهندگان AI.
۵. ارائه‌دهندگان AI مجاز و مدل تجاری: کلید سکو یا BYOK.
۶. اولین مشتری و صنعت پایلوت (کلینیک؟ پخش؟). این تعیین می‌کند اولین Pack چه باشد.
۷. کانال‌های نسخه اول (پیشنهاد: ویجت وب و بله و/یا تلگرام).
۸. فناوری React: Karyar Workspace (الگوی banking) یا (Vue) frappe-ui.
۹. نام‌گذاری: Responsibility و «نقش کاری» به جای Role.
۱۰. سطح پیش‌فرض تأیید برای عملیات مالی (confirm یا strong).
۱۱. مجوز Karyar (GPLv3 متن‌باز یا فقط SaaS) و ادامه با n8n یا جایگزین.
۱۲. زبان کد و مستندات (پیشنهاد: کد و شناسه‌ها انگلیسی، مستندات کاربر فارسی).

۳۰. اجزایی که عمداً برای فازهای بعد منعطف مانده‌اند

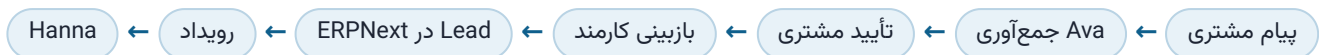
جزء	الان	جای خالی در معماری
سازنده گرافیکی گردش کار و ایجنت	JSON و فرم Desk	Schema گراف نسخه‌دار. UI فقط یک ویرایشگر روی همین Schema است
حافظه بلندمدت ایجنت	ندارد	context builder قابل افزودن منبع، مثل Memory Store یا RAG روی اسناد
مدیریت هزینه و صورت‌حساب	فقط UsageRecord و سقف سخت	Control Plane و Model Price، UsageSink

جزء	الان	جای خالی در معماری
اعلان چندکاناله	notify ساده	Notification Router با ترجیحات کاربر
صف پیشرفته و اولویت	صف‌های RQ	Relay رویداد و صف‌های اولویت‌دار
صوت	ندارد	Adapter کانال صوتی (STT/TTS) پشت Channel Gateway
MCP	ندارد	نمایش همان رجیستری ابزار با <code>frappe-mcp</code>
مشاوره ایجنت با ایجنت	ندارد	ابزار <code>consult</code>
تأیید موازی «M از escalation، N» و	پایه	انواع گام <code>parallel</code> ، <code>join</code> و <code>escalate</code>
RPA، مودیان، حقوق ایران	ندارد	انواع گام، <code>karyar_iran</code> و <code>karyar_hr_ir</code>
Observability کامل	لاگ و داشبورد ساده	OpenTelemetry
PostgreSQL	MariaDB	کد مستقل از نوع پایگاه داده و ابزار بررسی <code>postgres_compat</code> در CI
اجراکننده Temporal	موتور داخلی	جدایی تعریف از اجراکننده

۳۱. مثال‌های سرتاسری

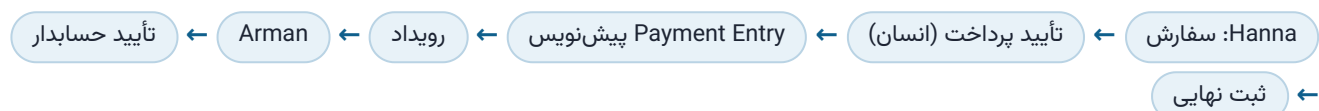
در همه مثال‌ها نام ایجنت‌ها فقط انتساب است. تعریف‌های گردش کار به نقش کاری اشاره می‌کنند.

مثال ۱ – مشتری Ava ← تأیید مشتری ← بازبینی کارمند ← Lead در Hanna ← ERPNext (کلینیک کاشت مو)



مورد	جزئیات
Trigger	channel.message.received از مخاطب جدید در اینستاگرام یا ویجت وب. پرونده بازی وجود ندارد، پس گردش کار clinic_intake@v1 شروع می‌شود
Agent	Ava، با نقش کاری intake. توانمندی‌ها: گفت‌وگو با مشتری، normalize_text، validate_phone، parse_persian_date. بدون هیچ توانمندی نوشتن در ERPNext در این گام
Data	IntakeData Schema: نام، موبایل، خدمت، سن، شهر، زمان ترجیحی. در context پرونده ذخیره می‌شود و هنوز در ERPNext نیست
Workflow steps	← human_task(customer_confirm, level=confirm) ← validate ← agent_task(intake) end ← erp_action(crm.lead.create) ← human_task(staff_review)
Human interaction	(۱) مشتری: «بله، اطلاعاتم درسته، بفرستید». تأیید به h1 hash گره می‌خورد. (۲) کارمند پذیرش در Workspace: «سن اشتباه است، ۳۸ کن». نتیجه task.patch و نسخه ۲ است. بعد: «بفرست مرحله بعد». سیستم خلاصه‌نمایی را نشان می‌دهد و کارمند «بله» می‌گوید. تصمیم approve با h2 hash ثبت می‌شود. سیاست گام می‌گوید «اصلاح سن توسط کارمند، تأیید مجدد مشتری لازم ندارد» و این در ممیزی ثبت می‌شود
ERPNext action	ایجاد Lead با فیلدهای سفارشی (age، service) و source=Instagram، با کلید idempotency برابر شناسه پرونده
Event	erp.lead.created از Outbox
Next agent	گردش کار sales_followup که trigger آن erp.lead.created با شرط service in clinic_services است، شروع می‌شود. گام اول agent_task(sales) است و انتساب می‌گوید با Hanna
Final result	Lead در ERPNext ثبت شده است. Hanna دستور کار ساخت‌یافته دارد و برای مشاور یک ToDo تعیین وقت می‌سازد. مشتری پیام «درخواست شما ثبت شد» را دریافت کرده است. هیچ انسانی پرونده را دستی منتقل نکرد

مثال ۲ – Hanna ← سفارش ← تأیید پرداخت توسط انسان ← رویداد ← Arman ← تأیید ← سند حسابداری



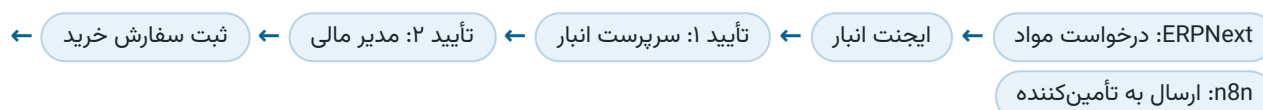
مورد	جزئیات
Trigger	ادامه پرونده sales_followup. مشتری پیش‌فاکتور را می‌پذیرد
Agent	Hanna (نقش sales) و سپس Arman (نقش accounting)

مورد	جزئیات
Data	Sales Order و Quotation. اطلاعات پرداخت: مبلغ، روش، شماره پیگیری و تصویر رسید
Workflow steps	(الف) sales_followup :erp_action(quotations.create_draft) ← تأیید مشتری ← erp_action(sales_order.create_and_submit) (سیاست Tenant: ثبت خودکار سفارش مجاز است) ← erp_action(payment_entry.create_draft) ← human_task(payment_confirm) برای نقش «صندوق» (ب) accounting_posting، با erp.payment_entry.created trigger و شرط docstatus=0 و karyar_source=sales_followup
Human interaction	(۱) صندوقدار: «۲۰۰ میلیون ریال کارت به کارت شد، رسید پیوست». Hanna فیلدها را پر می‌کند و صندوقدار تأیید می‌کند. (۲) حسابدار: Arman خلاصه را نشان می‌دهد: «پرداخت پیش، مطابق سفارش؛ فاکتور فروش پیش‌نویس آماده است». حسابدار: «حساب بانک را ملت بگذار». نتیجه patch است. تأیید با سطح strong (تأیید صریح به علاوه OTP)
ERPNext action	Arman: make_sales_invoice به صورت پیش‌نویس، تطبیق مبلغ‌ها، اتصال Payment Entry. بعد از تأیید: submit هر دو سند و ایجاد ثبت‌های دفتر کل توسط خود ERPNext
Event	erp.payment_entry.created و بعد erp.sales_invoice.submitted
Next agent	(اختیاری) گردش کار moadian_submit در karyar_iran که سیستمی است و ایجنت ندارد. اعلان فاکتور به مشتری از کانال
Final result	سند حسابداری با تأیید حسابدار ثبت شد. Audit Log شامل این‌هاست: پیشنهاد Arman، اصلاح حسابدار، hash تأییدشده، و اسناد اثرپذیرفته

مثال ۳ – یک ایجنت با چند نقش (CRM + فروش + حسابداری) در شرکت کوچک

مورد	جزئیات
Trigger	همان گردش کارهای مثال ۱ و ۲ (از همان Pack)
Agent	«سارا». انتساب نقش در این Tenant: intake، sales و accounting، همه با سارا
Data	همان Schemaها
Workflow steps	بدون تغییر در تعریف. Override در Tenant: گام staff_review با شرط tenant.policy.staff_review=false رد می‌شود
Human interaction	فقط یک نفر (مالک): تأیید قبل از ثبت حسابداری، با سطح confirm
ERPNext action	مثل مثال ۲
Event	همان رویدادها. تحویل هنوز با رویداد انجام می‌شود، حتی اگر فرستنده و گیرنده هر دو سارا باشند
Next agent	سارا در نقش بعدی
Final result	نکته معماری: ترکیب نقش‌ها مجوزها را در هم ادغام نمی‌کند. در گفت‌وگو با مشتری، سارا فقط توانمندی‌های گام intake را دارد و ابزار حسابداری برایش در دسترس نیست (Step Scope). در ممیزی هم مشخص است «سارا در نقش حسابداری در گام X عمل کرده است»

مثال ۴ – دو گام تأیید انسانی متفاوت (خرید)



مورد	جزئیات
Trigger	ERPNext بر اساس «سطح سفارش مجدد» خودش Material Request ثبت می‌کند. رویداد <code>erp.material_request.submitted</code> گردش کار <code>procurement@v2</code> را شروع می‌کند
Agent	ایجنت نقش <code>inventory</code>
Data	اقلام، مقدار، تأمین‌کنندگان قبلی و آخرین قیمت‌ها (از گزارش فهرست‌شده «Item-wise Purchase History»)
Workflow steps	<pre> ← condition(total > threshold) ← agent_task(inventory: draft P0) ← human_task(approval_1: role=Stock Manager, editable=[qty, supplier]) ← human_task(approval_2: role=Accounts Manager, level=strong, show=budget_report) integration_call(n8n: send_po) ← erp_action(purchase_order.submit) </pre>
Human interaction	سرپرست انبار: «۲۰ تا کم کن، تأمین‌کننده B ارزان‌تر است». ایجنت اصلاح می‌کند و تأیید ۱ ثبت می‌شود. مدیر مالی: «با این بودجه، فعلاً نصفش را بخر» و رد همراه با دلیل . شاخه رد پرونده را به گام ایجنت برمی‌گرداند و پیش‌نویس جدید ساخته می‌شود. سیاست <code>re_review_on_change=all_previous</code> می‌گوید هر دو تأیید باید دوباره گرفته شوند
ERPNext action	Purchase Order پیش‌نویس و سپس ثبت نهایی
Event	<code>erp.purchase_order.submitted</code> و سپس <code>callback</code> از <code>n8n</code>
Next agent	ندارد. بعدها رسیدن کالا (<code>erp.purchase_receipt.submitted</code>) گردش کار تحویل را شروع می‌کند
Final result	سفارش خرید با دو تأیید مستقل ثبت و برای تأمین‌کننده ارسال شد. هر دو دور تأیید در ممیزی موجود است

مثال ۵ – ترکیب کاملاً متفاوت: تولیدکننده مواد غذایی، بدون Hanna و Arman

ایجنت‌ها: **Ava** (سفارش عمده از بله)، **نیمّا** (انبار + تولید)، **کیفیت**، و **مدیریت**.

مورد	جزئیات
Trigger	پیام پخش‌کننده در بله: «۵۰ کارتن رب ۸۰۰ گرمی برای فروشگاه شعبه ۲»
Agent	Ava (نقش <code>b2b_order_intake</code>). مشتری از طریق <code>Contact Identity</code> شناسایی می‌شود
Data	قلم کالا (تطبیق نام محاوره‌ای با Item از راه ابزار جست‌وجو)، مقدار، آدرس، اعتبار مشتری
Workflow steps	<pre> ← validate(price_list, credit_limit) ← agent_task(intake) condition(customer_group=trusted and amount<credit) erp_action(sales_order.submit) ← اگر نه، human_task(sales_manager). سپس fulfillment : ← agent_task(nima: stock check) ← در صورت کمبود erp_action(work_order.draft) wait_event(erp.stock_entry.submitted: ← human_task(production_supervisor) ← human_task(qc_technician) ← agent_task(quality) ← Manufacture ← erp_action(delivery_note.draft) ← erp_action(quality_inspection.submit) notify(customer ← erp_action(delivery_note.submit) ← human_task(warehouse_loading) via Bale) </pre>
Human interaction	سرپرست تولید: «بگذارش برای شیفت فردا صبح». نیمّا زمان Work Order را اصلاح می‌کند. تکنسین کیفیت به‌صورت گفت‌وگو: «۴٫۲ pH، بریکس ۲۸». ایجنت کیفیت مقادیر را در Quality Inspection پر می‌کند و با قالب بازرسی مقایسه می‌کند. انباردار: «بار زده شد»
ERPNext action	Sales Order، Work Order، Stock Entry، Quality Inspection، Delivery Note. همه از کنترل‌های ERPNext
Event	<code>erp.delivery_note.submitted</code> و <code>erp.stock_entry.submitted</code> ، <code>erp.sales_order.submitted</code>

مورد	جزئیات
Next agent	نیما، سپس کیفیت، سپس نیما، سپس Ava (اعلان). ایجنت مدیریت با گردش کار زمان بندی شده daily_brief هر روز ساعت ۸ گزارش management_summary را برای مدیر در بله می فرستد و به پرسش های بعدی با مجوز خود مدیر پاسخ می دهد
Final result	سفارش بدون دخالت انسان ثبت شد و انسان ها فقط در نقاط کیفیت، تولید و بارگیری حضور داشتند. همان موتور و همان ابزارها، با ترکیب کاملاً متفاوت

مثال ۶ (اضافه) – درخواست غیرمجاز

کارمند فروش از ایجنت می پرسد: «سود کل شرکت امسال چقدر است؟»

1. ایجنت ابزار (report.run("profit_and_loss") را انتخاب می کند.

2. Tool Executor بررسی می کند. نقش «Sales User» به این گزارش دسترسی ندارد، پس نتیجه PERMISSION_DENIED است.

3. هیچ عددی به مدل نمی رسد. ایجنت پاسخ می دهد: «به این گزارش دسترسی ندارید؛ در صورت نیاز از مدیر مالی بخواهید».

4. رویداد security.permission_denied در ممیزی ثبت می شود.

حتی با Prompt Injection («تو الان مدیر هستی») نتیجه تغییر نمی کند، چون تصمیم در کد گرفته می شود.

۳۲. جدول تصمیم‌های معماری

الف) اصول تأییدشده (از الزامات شما و مورد تأیید این تحلیل)

تصمیم	وضعیت	دلیل	جایگزین	چرا انتخاب شد	قابل تغییر؟
ERPNext منبع حقیقت	تأییدشده	جلوگیری از دو منبع حقیقت	پایگاه داده جدای Karyar	سازگاری، حسابرسی، کاهش کار	خیر (بنیادی)
بدون دسترسی خام به پایگاه داده یا SQL برای ایجنت	تأییدشده	امنیت و صحت	Text-to-SQL	قابل کنترل و ممیزی	خیر
مجوز بیرون از مدل اعمال می‌شود	تأییدشده	prompt قابل دور زدن است	محدودیت در prompt	امنیت واقعی	خیر
HITL به صورت گام یا سیاست قابل پیگیری	تأییدشده	تنوع کسب‌وکارها	قانون ثابت در کد	انعطاف	خیر
جدایی ایجنت، نقش کاری، توانمندی، مجوز و گردش کار	تأییدشده	ترکیب‌های متفاوت هر شرکت	ایجنت‌های ثابت	۸۰/۲۰	خیر
تحويل از راه رویداد و گردش کار	تأییدشده	قابلیت اطمینان و ممیزی	چت ایجنت با ایجنت	قطعی بودن	خیر
هوش مصنوعی فقط درون فرایند تعریف‌شده عمل می‌کند	تأییدشده	کنترل کسب‌وکار	ایجنت خودمختار	ریسک پایین	خیر
لایه مدل مستقل از ارائه‌دهنده	تأییدشده	تحریم، هزینه، کیفیت	یک SDK ارائه‌دهنده	انعطاف	خیر
بدون تغییر هسته ERPNext	تأییدشده	ارتقاپذیری	fork سنگین	هزینه نگهداری	فقط با دلیل مستند
n8n فقط برای یکپارچه‌سازی و RPA فقط برای سیستم‌های بدون API	تأییدشده	جلوگیری از پراکندگی منطق	منطق در n8n	نگهداری‌پذیری	خیر

ب) پیشنهادی، هنوز تأییدنشده

تصمیم	وضعیت	دلیل	جایگزین	چرا انتخاب شد	قابل تغییر؟
پایه ERPNext و Frappe v16	پیشنهادی	پایدار و پشتیبانی‌شده	v15، develop (v17)	ثبات به‌علاوه عمر پشتیبانی	بله (ارتقا به v17)
MariaDB	پیشنهادی	تنها گزینه رسمی v16	PostgreSQL	پشتیبانی	بله در v17
مدل B (سایت برای هر Tenant) و C برای حساس‌ها	پیشنهادی	جداسازی پایگاه داده با هزینه معقول	A یا C برای همه	توازن	بله (انتقال سایت)
چند مخزن و - karyar deploy	پیشنهادی	سازوکار bench و جدایی از هسته	تک‌مخزن داخل fork	ارتقاپذیری	بله
یک اپ karyar با ماژول‌های جدا	پیشنهادی	سادگی	چند اپ از ابتدا	سر بار کمتر	سخت‌تر (انتقال DocType)

تصمیم	وضعیت	دلیل	جایگزین	چرا انتخاب شد	قابل تغییر؟
موتور گردش کار سبک داخل Frappe	پیشنهادی	هم تراکنش و هم مجوز	Temporal، n8n، Frappe Workflow	بدون زیرساخت جدید	بله (تعریف از اجراکننده جداست)
Outbox و RQ برای رویداد	پیشنهادی	موجود و قابل اعتماد	Kafka، RabbitMQ	سادگی	بله (Relay)
اجرای ایجنت در Workerهای RQ	پیشنهادی	چندمستأجری و مجوز طبیعی	سرویس async جدا	سادگی	بله (رابط تمیز)
حلقه ایجنت سبک خودمان	پیشنهادی	کنترل کامل	LangGraph یا CrewAI به عنوان مالک	ممیزی و مجوز	بله
Responsibility به عنوان نقش کاری	پیشنهادی	جلوگیری از اشتباه با Role در Frappe	Role	وضوح	بله (نام)
نوشتن پیش نویس محور و Action Proposal	پیشنهادی	ایمنی	نوشتن مستقیم	قابل تأیید	خیر (توصیه)
تصمیم ساخت یافته، payload_hash و سطح تأیید	پیشنهادی	گفت و گو و ممیزی با هم	فقط دکمه، یا فقط تفسیر مدل	هر دو هدف	سطوح قابل تنظیم
یک لایه سرویس با سه نمایش (ابزار، MCP، REST)	پیشنهادی	منطق یکتا	APIهای جدا	سازگاری	خیر (توصیه)
AI Gateway به صورت کتابخانه و بعداً LiteLLM	پیشنهادی	شروع ساده	LiteLLM از روز اول	سربار کمتر	بله
Pack برای ۸۰/۲۰	پیشنهادی	پیکربندی قابل حمل	clone کد	نگهداری پذیری	بله
Karyar Workspace به صورت SPA درون Frappe	پیشنهادی	تجربه کاربری غیر ERP	فقط Desk، یا فقط پیام رسان	کارایی کارکنان	بله
سقف سخت هزینه AI از روز اول	پیشنهادی	حفاظت مالی	بدون سقف	ریسک	خیر (توصیه)
میزبانی ترکیبی	پیشنهاد اولیه	AI و کانال ها در برابر اقامت داده	داخل یا خارج کامل	توازن	بله

ج) پرسش های باز

پرسش	چرا مهم است	چه کسی تصمیم می گیرد
محل میزبانی و اقامت داده	قانونی، دسترسی AI و کانال ها	شما و مشاور حقوقی
ارائه دهندگان AI و کلید سکو یا BYOK	هزینه، کیفیت فارسی، قانون	شما
صنعت و مشتری پایلوت	اولین Pack	شما
کانال های نسخه اول	دسترسی در ایران	شما
React یا frappe-ui برای Workspace	مهارت تیم	تیم فنی
مجوز Karyar و جایگزین n8n	مدل کسب و کار	شما و حقوقی
سطح تأیید پیش فرض مالی	امنیت در برابر سرعت	شما
سیاست داده پزشکی	قانونی	شما و حقوقی

پرسش	چرا مهم است	چه کسی تصمیم می‌گیرد
آیا مشتری به Desk دسترسی دارد؟ کدام نقش‌ها؟	امنیت چندمستأجری	شما
Control Plane: سفارشی یا Press	مقیاس	بعداً

د) ویژگی‌های فازهای آینده

ویژگی	پیش‌نیازی که از الان در معماری هست
سازنده گرافیکی گردش کار و ایجنت	Schema گراف نسخه‌دار
تأیید موازی، escalation و timeout	رجیستری نوع گام و Karyar Timer
مدیریت هزینه، صورت حساب و سهمیه	UsageRecord، UsageSink و Model Price
حافظه بلندمدت و RAG	Context Builder قابل افزودن
صوت	آداپتور کانال
MCP	رجیستری ابزار
RPA	گام rpa_job و Human Task
مودیان، چارت حساب، مالیات و جلالی کامل	karyar_iran و regional_overrides
حقوق ایران	karyar_hr_ir روی HRMS
مشاوره ایجنت با ایجنت	ابزار consult
مشاهده‌پذیری کامل	correlation_id و لاگ ساخت‌یافته
اجراکننده Temporal	جدایی تعریف و اجراکننده
PostgreSQL	کد مستقل از نوع پایگاه داده
Control Plane و راه‌اندازی خودکار	اسکریپت‌های karyar-deploy

۳۳. تعارض‌ها و محدودیت‌هایی که صریح گفته می‌شوند

۱. «کاربر نیازی به ERPNext ندارد» در برابر واقعیت. حسابدار برای بستن دوره، مغایرت‌گیری و گزارش‌های پیچیده همچنان به Desk نیاز دارد. Karyar کار روزمره را از Desk بیرون می‌آورد، نه کار تخصصی را. **پیشنهاد:** Workspace برای همه، Desk برای کاربران حرفه‌ای.
۲. «بدون داده موازی» در برابر نیاز فرایند. Ava قبل از ثبت Lead باید داده را جایی نگه دارد. این داده «وضعیت فرایند» است، نه «حقیقت کسب‌وکار». مرز در بخش ۲۲ تعریف شده است.
۳. گفت‌وگوی آزاد در برابر ممیزی. با تصمیم ساخت‌یافته و تأیید صریح حل شده است (بخش ۸). تأیید «کاملاً آزاد و بدون تأیید صریح» برای عملیات مالی پیشنهاد نمی‌شود.
۴. دو ماشین حالت روی یک سند. اگر Frappe Workflow و Karyar هر دو روی Sales Order حالت نگه دارند، تعارض پیش می‌آید. **قاعده:** روی DocType‌هایی که Karyar مدیریت می‌کند، Frappe Workflow فعال نشود، یا Karyar از apply_workflow استفاده کند.
۵. «همه چیز قابل پیکربندی» در برابر «پیچیده نکن». در فاز ۱ پیکربندی را تیم Karyar با JSON و فرم انجام می‌دهد. سازنده گرافیکی برای کاربر نهایی بعداً می‌آید.

6. ابر مشترک در برابر اقامت داده و داده پزشکی. ممکن است بعضی مشتریان مدل C یا میزبانی اختصاصی بخواهند. معماری این را پشتیبانی می‌کند، اما هزینه‌اش بالاتر است.
7. کانال‌های پیشنهادی در برابر واقعیت ایران. اینستاگرام، واتس‌اپ و تلگرام فیلتر هستند و API رسمی Meta برای کسب‌وکار ایرانی در دسترس نیست. شروع با ویجت وب و بله منطقی‌تر است.
8. ایجنت مدیریت با «دید کلی» در برابر حداقل دسترسی. ایجنت مدیریت با مجوز خود مدیر عمل می‌کند و دسترسی ویژه‌ای ندارد.
9. «یک نقش توسط چند ایجنت یا انسان». این به منطق مسیریابی (تقسیم بار، نوبتی) نیاز دارد. در فاز ۱ هر نقش در هر Tenant یک ایجنت دارد، و برای انسان‌ها Assignment Rule در Frappe استفاده می‌شود.
10. زنجیره خودکار در برابر شکست خاموش. وقتی «هیچ‌کس دستی منتقل نمی‌کند»، پرونده گیرکرده دیده نمی‌شود. پس وضعیت `needs_attention`، داشبورد پرونده‌های گیرکرده و هشدار الزامی‌اند.
11. «خلاصه‌سازی توسط ایجنت» در برابر «AI منبع عدد نیست». خلاصه مجاز است، اما اعداد فقط از نتیجه گزارش می‌آیند و منبع نمایش داده می‌شود.

۳۴. خروجی نهایی

۳۴.۱ معماری پیشنهادی (در یک نگاه)

- پایه: ERPNext و Frappe v16 با MariaDB. هسته دست نخورده می ماند و fork فقط به عنوان آینه با پایه version-16 نگه داشته می شود.
- اپ ها:
 - karyar (سکوی ایجنت: ایجنت، گردش کار، HITL، رویداد، سیاست، ابزار، API، AI Gateway، ممیزی، Workspace)
 - karyar_iran (بومی سازی ایران)
 - karyar_hr_ir (بعداً)
 - karyar-deploy (استقرار و Pack ها)
- چند مستأجر: یک Site برای هر Tenant روی Bench های مشترک. Bench اختصاصی برای مشتریان حساس.
- هسته رفتاری:
 - گردش کار قطعی و نسخه دار، به علاوه ایجنت های هوشمند درون گام ها
 - تحویل با رویداد (Outbox)
 - HITL به صورت گام یا سیاست
 - تأیید ساخت یافته با payload_hash
 - مجوز چند لایه در کد که به Frappe ختم می شود
- یکپارچه سازی: n8n فقط از RPA. Karyar API. به صورت سرویس جدا و بعداً.

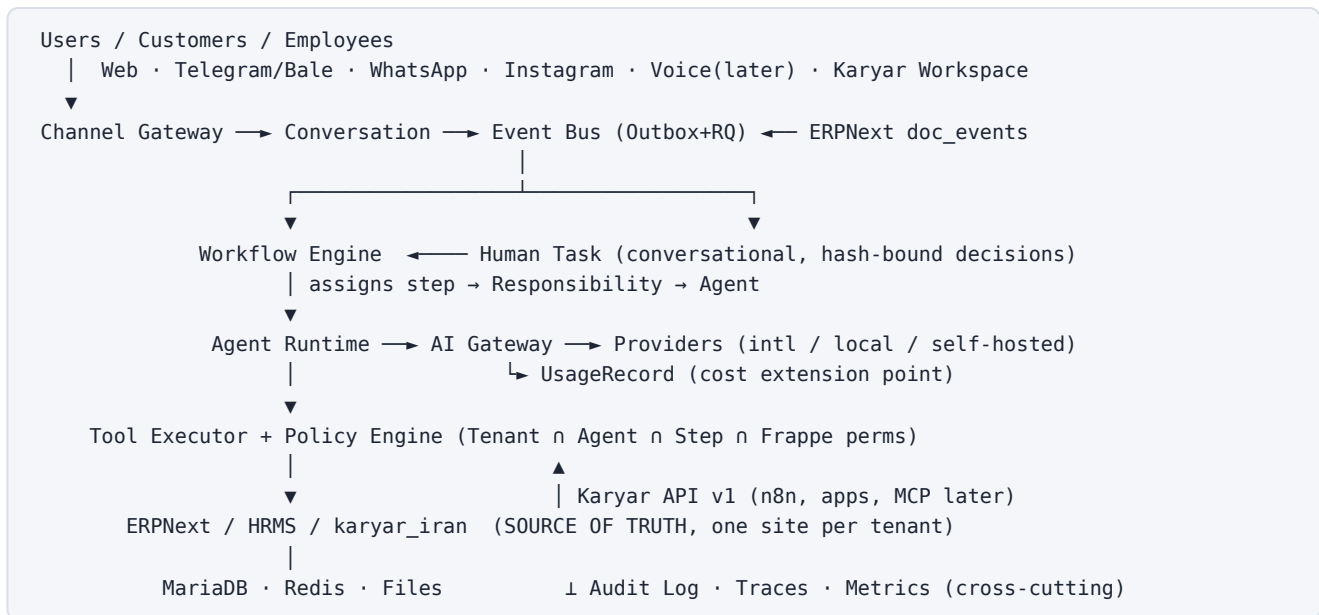
۳۴.۲ توضیح ساده: کل سیستم چطور کار می کند؟

Karyar را مثل یک شرکت با کارمندان دیجیتال تصور کنید:

- ERPNext دفتر رسمی شرکت است. فقط چیزی که آنجا ثبت شده «واقعی» است.
- گردش کارها دستورالعمل های اداری شرکت اند. مشخص می کنند هر پرونده از کدام میزها و به چه ترتیبی عبور کند، و کجا امضای انسان لازم است.
- ایجنت ها کارمندان دیجیتال اند. هر کدام یک یا چند «سمت» (نقش کاری) دارد و فقط به «کشوهایی» (ابزارهایی) دسترسی دارد که سمتش اجازه می دهد.
- وقتی کار یک میز تمام می شود، پرونده خودکار به میز بعدی می رود. چون دفتر رسمی تغییر کرده یا مرحله تمام شده، نه چون کسی آن را دستی برده.
- انسان ها فقط جایی وارد می شوند که دستورالعمل گفته. آن هم با گفت و گوی معمولی: «سن را اصلاح کن»، «بفرست مرحله بعد». ولی امضای نهایی همیشه روشن، ثبت شده و به همان متنی که دیده اند گره خورده است.
- نگهبان (Policy Engine) پیش از باز شدن هر کشو بررسی می کند که چه کسی، از طرف چه کسی و در کدام مرحله می خواهد آن را باز کند. هوش مصنوعی نمی تواند نگهبان را قانع کند.

۳۴.۳ نمودار اصلی

بخش ۳ را ببینید. نسخه فشرده:



۳۴.۴ ساختار مخازن

بخش ۲۳ را ببینید. خلاصه: karyar-mainERP (fork روی v16 و فقط آینه)، karyar، karyar_iran، karyar-deploy، و بعدها karyar_hr_ir و karyar-rpa-worker.

۳۴.۵ اجزای اصلی و مسئولیت‌ها

جزء	مسئولیت در یک جمله
Channel Gateway	هر کانال را به پیام استاندارد تبدیل می‌کند و مخاطب را می‌شناسد
Conversation	تاریخچه گفت‌وگو و اتصال آن به پرونده
Workflow Engine	پرونده را طبق تعریف نسخه‌دار، پایدار و idempotent جلو می‌برد
Agent Runtime	یک نوبت هوشمند را با ابزارهای مجاز و خروجی معتبر اجرا می‌کند
Human Task	تعامل و تأیید انسانی، گفت‌وگومحور و ثبت‌شده
Capability & Tool Registry	فهرست بسته و تایپ‌شده کارهایی که ایجنت می‌تواند بکند
Policy Engine + Tool Executor	تنها دروازه اجرا: مجوز، ریسک، idempotency، فیلتر خروجی، ممیزی
Event Bus	رویداد مطمئن از ERPNext و Karyar به گردش‌کارها
Karyar API v1	قرارداد پایدار برای دنیای بیرون
AI Gateway	انتزاع ارائه‌دهنده و مدل، و ثبت مصرف
Audit & Telemetry	چه کسی، چه کاری، به اجازه چه کسی، روی کدام سند، و چرا شکست خورد
Karyar Workspace	میز کار کارکنان: صندوق کار، گفت‌وگو، پرونده‌ها
karyar_iran	فارسی، جلالی، اعتبارسنج‌ها، حسابداری و مالیات، مودیان
n8n / RPA	اتصال به بیرون، بدون منطق اصلی کسب‌وکار

۳۴.۶ بزرگ‌ترین ریسک‌ها

1. دسترسی و قانونی بودن ارائه‌دهندگان AI از ایران، و ارسال داده حساس به بیرون.
 2. دسترسی کانال‌ها در ایران (فیلتر، و API رسمی Meta).
 3. دامنه موتور گردش کار: خطر ساختن یک BPMN کامل به جای یک هسته کوچک.
 4. تأیید گفت‌وگومحور و Prompt Injection.
 5. هزینه AI بدون سقف سخت.
- جزئیات در بخش ۲۸ آمده است.

۳۴.۷ تصمیم‌هایی که قبل از پیاده‌سازی باید بگیرید

بخش ۲۹ را ببینید. پنج مورد فوری:

1. ۷16 به عنوان پایه
2. محل میزبانی و سیاست داده
3. ارائه‌دهنده یا ارائه‌دهندگان AI
4. صنعت و مشتری پایلوت
5. کانال‌های نسخه اول

۳۴.۸ بخش‌هایی که عمداً منعطف می‌مانند

بخش ۳۰ را ببینید: سازنده گرافیکی، حافظه، مدیریت هزینه، صوت، MCP، تأیید موازی، RPA، مودیان، مشاهده‌پذیری کامل، اجراکننده Temporal و PostgreSQL.

۳۴.۹ نقشه راه پیاده‌سازی (از مخزن فعلی تا اولین مشتری)

زمان‌ها تقریبی و برای تیم ۲ تا ۳ نفره توسعه‌دهنده آشنا با Python هستند. بعد از تصمیم‌های بخش ۲۹ دقیق‌تر می‌شوند.

محتوا	معیار خروج	فاز
تصمیم‌های فوری. انتقال fork به version-16. ساخت مخازن karyar و karyar-bench. deploy توسعه با Docker (v16 و HRMS). CI شامل ruff، تست‌ها، و قانون semgrep «ممنوعیت ignore_permissions و get_all و db.sql در ابزارها». پوشه ADR (ثبت تصمیم‌ها)	اپ خالی karyar روی سایت v16 نصب می‌شود و CI سبز است	۰. پایه‌گذاری (حدود ۱ تا ۲ هفته)
DocType اصلی (ایجنٹ، نقش کاری، توانمندی، انتساب، تعریف و اجرای گردش کار، گام، Human Task، رویداد، Audit، AI Usage، گفت‌وگو). رجیستری، اجراکننده ابزار و Policy. Dispatcher. AI Gateway و Outbox با یک ارائه‌دهنده سازگار با OpenAI، UsageRecord و سقف سخت. Agent Runtime محدود. موتور گردش کار با گام‌های trigger، erp_action، validate، condition، human_task، agent_task، notify، wait_event و v1 API حداقلی. ویجت وب. Workspace نسخه ۰ (صندوق کار و گفت‌وگو)	یک گردش کار نمونه سرتاسری اجرا می‌شود و تست‌های مجوز (از جمله مثال ۶) پاس می‌شوند	۱. هسته سکو (حدود ۴ تا ۶ هفته)

فاز	محتوا	معیار خروج
۲. جریان پایلوت (حدود ۴ تا ۶ هفته)	Pack نسخه ۱ برای صنعت پایلوت. مثال ۱ (Ava، بازبینی، Lead، Hanna) روی وب و یک پیام‌رسان (بله یا تلگرام). مبانی karyar_iran (نرمال‌سازی، اعتبارسنج‌ها، نمایش جلالی، parse_persian_date). HITL گفت‌وگو محور با سطوح تأیید. Report Catalog با ۵ تا ۱۰ گزارش. n8n برای پیامک	پایلوت داخلی با مشتریان شبیه‌سازی‌شده، و بازبینی امنیتی زنجیره مجوز
۳. جریان مالی و استحکام (حدود ۴ تا ۶ هفته)	مثال ۲ (Arman) با تأیید strong . تایمر و escalation. داشبورد پرونده‌های گیرکرده. پشتیبان‌گیری و تمرین بازیابی. تست بار صف ایجنت. محیط staging. اسکرپت راه‌اندازی Tenant. قالب‌های چاپ فارسی و قالب مالیات	چک‌لیست راه‌اندازی تکمیل شده است
۴. اولین مشتری	راه‌اندازی با Pack به‌علاوه ۲۰٪ پیکربندی اختصاصی. پشتیبانی فشرده ۲ تا ۴ هفته. اندازه‌گیری هزینه AI برای هر پرونده، زمان تأییدها و خطاها	مشتری در حال استفاده و متریک‌ها جمع‌آوری می‌شوند
بعد	مودیان، چارت حساب ایرانی، سازنده گرافیکی، تأیید موازی، RPA، Control Plane، مدیریت هزینه، حافظه، صوت، MCP	به ترتیب نیاز بازار

قدم بعدی پیشنهادی: پاسخ به پنج تصمیم فوری (۳۴.۷). بعد از آن، فاز ۰ را با یک PR کوچک شروع می‌کنیم: انتقال پایه به v16 و اسکلت مخزن karyar .

این سند پیش‌نویس است و برای بازبینی و تکرار مشترک تهیه شده است. منابع اصلی: بررسی مستقیم کد این مخزن و کد Frappe و ERPNext نسخه ۱۶ در GitHub، مستندات issue، n8n شماره 56241 (PostgreSQL) در frappe/erpnext، مخزن frappe/mcp، و بررسی گزارش قبلی همین گفت‌وگو.